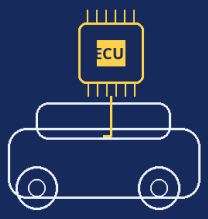
AUTOMOTIVE SOFTWARE DIAGNOSTICS

COMPREHENSIVE TECHNICAL HANDBOOK

UDS • ISO 14229 • OBD-II • AUTOSAR DCM/DEM • Fault Management

DID DTC RID SID PID

Vehicle + ECU



DTCs | Live Data | Flashing

Diagr Fault detection | Reaction | Service

OBD-II

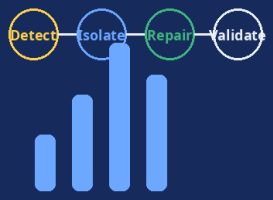
UDS

AUTOSAR DCM/DEM

DoIP / CAN

Request -> Response -> Reaction

Diagnostics Lifecycle



Fault detection | Reaction | Service

What the handbook covers

- **27+ UDS services**
  Session control, security, flashing, routines
- **OBD-II modes & PIDs**
  Emission monitoring, readiness, live data
- **AUTOSAR DCM/DEM**
  Fault events, DTC handling, inhibition
- **Cybersecurity & SOVD**
  Authentication, DoIP, modern vehicle diagnostics

Diagnostic coverage

Ashish Kumar
Senior Technical Leader & Associate Manager
KPIT Technologies Limited, Bengaluru
B.Tech. (EEE) - GCE Gaya • M.Tech (IIT ISM Dhanbad) • PG Certificate IIM Nagpur

UDS • OBD-II • AUTOSAR • DoIP • Safety • Cybersecurity

Edition: June 2026 | 21 Chapters + Appendices
ISO 14229:2020+ • AUTOSAR R24-11 • SAE J1979-2:2022 • ISO 21434:2021

PREFACE

This June 2026 edition is arranged as a practical reference for automotive software diagnostics, from first principles through UDS, OBD-II, AUTOSAR DCM/DEM, DoIP, bootloader programming, DIDs/RIDs, and cybersecurity.

The intent is simple: make the flow of diagnostics easier to follow for engineers working on ECUs, service tools, flashing, fault handling, and modern software-defined vehicles.

This handbook is designed to be read both linearly and selectively. Each chapter can also serve as a quick lookup guide during development, validation, or service integration.

Highlights

• UDS service flow, message formats, sessions, timing, security access, and negative responses

• OBD-II modes, PIDs, readiness monitors, and transport behavior over CAN

• AUTOSAR DCM, DEM, FiM, RTE, and diagnostic architecture

• DoIP, bootloader programming, SOVD, and cybersecurity implications

• DIDs, RIDs, flash sequences, EOL routines, and production diagnostics

CHAPTER 1

Introduction to Automotive Diagnostics

Automotive diagnostics is the systematic process of identifying, monitoring, and responding to faults within a vehicle's electronic control systems. As vehicles have evolved from purely mechanical systems to highly complex cyber-physical systems with over 100 Electronic Control Units (ECUs), software-based diagnostics has become fundamental to ensuring vehicle safety, regulatory compliance, and serviceability throughout the vehicle lifecycle.

1.1 Diagnostic Dimensions

Automotive software diagnostics spans three core dimensions:

- On-board diagnostics: real-time fault detection, DTC storage, and fault reaction by the ECU itself (AUTOSAR DEM/DCM).
- Off-board diagnostics: communication between a tester tool and ECU via a standardised protocol (UDS, OBD-II).
- Production diagnostics: end-of-line (EOL) calibration, ECU flashing, variant coding, and VIN programming.

1.2 Historical Evolution

- Pre-OBD Era (pre-1988): Proprietary manufacturer-specific diagnostic connectors. Mechanics relied on physical inspection.
- OBD-I (1988-1995): CARB-mandated basic emission monitoring for California vehicles. Inconsistent manufacturer implementations.
- OBD-II (1996+): SAE J1979 and EPA mandate. Unified 16-pin DLC (SAE J1962), standard PIDs, DTC formats. CAN from 2008.
- EOBD (2001+): European equivalent per EU Directive 98/69/EC, implemented in ISO 15031 series.
- WWH-OBD (2012+): World-Wide Harmonized OBD per UN ECE GTR No. 5. ISO 15765-based transport. Heavy-duty vehicles.
- UDS / ISO 14229 (2006+): Unified Diagnostic Services for in-depth ECU diagnostics, programming, and EOL testing.
- AUTOSAR Diagnostics Stack (2003+): Standardized DCM/DEM modules enabling portable diagnostic software.
- SOVD / HPC Era (2023+): Service-Oriented Vehicle Diagnostics (REST/HTTP) for High-Performance Computers in SDVs.

1.3 Regulatory and Standards Landscape

Standard	Region	Scope	Latest Version
OBD-II (SAE J1979 / ISO 15031)	USA (CARB)	Emission-related PIDs, standardised MIL	J1979-2:2022
EOBD (ISO 15031)	EU	Mirrors OBD-II for European vehicles	ISO 15031-5:2015
ISO 14229 (UDS)	Global	Full ECU service protocol (all sessions)	ISO 14229-1:2020
ISO 27145 (WWH-OBD)	Global	Heavy-duty worldwide harmonised OBD	ISO 27145-4:2012
ISO 13400 (DoIP)	Global	Diagnostics over Internet Protocol (Ethernet)	ISO 13400-2:2019

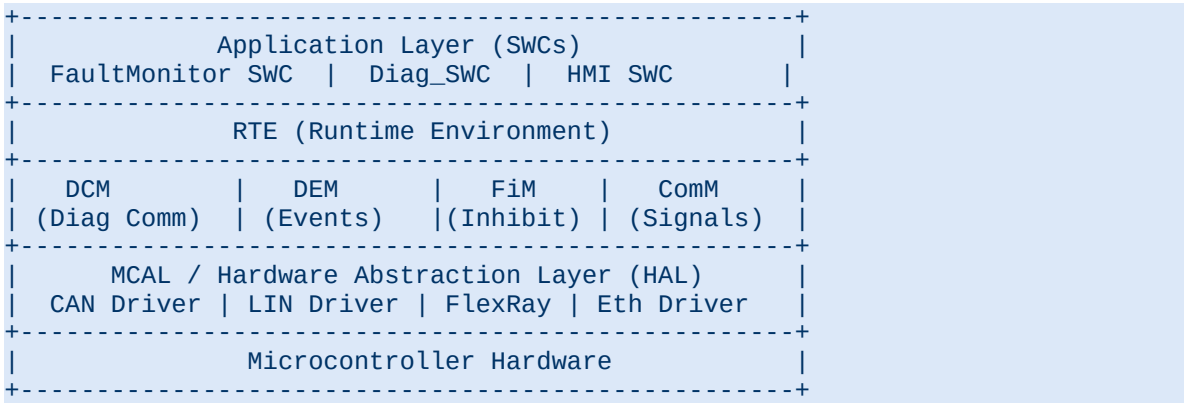
Standard	Region	Scope	Latest Version
AUTOSAR Classic/Adaptive	Global	SW stack for DCM, DEM, RTE	R24-11 (Nov 2024)
SAE J1939	USA/Global	Heavy-duty truck/bus diagnostics	J1939-73:2022
ISO 26262	Global	Functional Safety (impacts DEM/DCM design)	ISO 26262:2018
ISO 21434	Global	Cybersecurity engineering (impacts diag interfaces)	ISO 21434:2021
ASAM SOVD	Global	Service-Oriented Vehicle Diagnostics for HPCs	V1.2:2024

1.4 AUTOSAR Diagnostics Architecture

In an AUTOSAR-compliant ECU, the diagnostic stack consists of the following standardized modules:

Module	Full Name	Layer	Key Responsibility
DCM	Diagnostic Communication Manager	BSW	Handles UDS/OBD protocol services, session & security management
DEM	Diagnostic Event Manager	BSW	Fault detection, DTC storage, aging, healing, indicator control
FIM	Function Inhibition Manager	BSW	Inhibits SWC functions based on DEM event status
ComM	Communication Manager	BSW	Manages bus communication modes (Full/Silent/No)
PduR	PDU Router	BSW	Routes diagnostic PDUs between DCM and transport layers
CanTp	CAN Transport Layer	BSW	ISO-TP segmentation/reassembly over CAN (ISO 15765-2)
DoIP	Diagnostics over IP	BSW	ISO 13400 implementation for Ethernet-based diagnostics
RTE	Runtime Environment	RTE	Inter-module communication, port interfaces for DCM/DEM
SWC	Software Components	ASW	Application-level monitors, DID providers, routine handlers

Layered architecture diagram:



1.5 Physical and Network Layer Overview

Bus / Protocol	Speed	ISO Standard	Diagnostics Use
CAN 2.0B (Classical)	1 Mbps	ISO 11898	Baseline OBD-II; UDS on powertrain/body ECUs
CAN FD	8 Mbps data	ISO 11898-1:2015	Faster ECU flashing, extended payloads up to 64 bytes
Automotive Ethernet 100BASE-T1	100 Mbps	IEEE 802.3bw	ADAS, gateway, HPC, OTA

Bus / Protocol	Speed	ISO Standard	Diagnostics Use
DoIP over TCP/UDP	100+ Mbps	ISO 13400-2:2019	UDS over Ethernet; OEM dealer tools
K-Line / L-Line	12.4 kbps	ISO 14230	Legacy European ECUs (KWP2000), pre-2008
LIN	20 kbps	ISO 17987	LIN slave node diagnostics via master gateway
FlexRay	10 Mbps	ISO 10681-2	Safety-critical chassis ECUs (active suspension)

ISO-TP (ISO 15765-2) Transport Protocol

ISO-TP is the transport layer used to segment UDS messages exceeding 8 bytes over CAN. Four frame types: Single Frame (SF, 0x0N), First Frame (FF, 0x1N), Consecutive Frame (CF, 0x2N), and Flow Control (FC, 0x3N). Max PDU is 4095 bytes for Classical CAN and up to 2^32-1 bytes for CAN FD with extended addressing.

CHAPTER 2

Unified Diagnostic Services (UDS) — ISO 14229

ISO 14229 (UDS) is the central standard defining the application layer protocol for off-board communication with automotive ECUs. It specifies services, message formats, error handling, and timing requirements that enable diagnostic tools, testers, and flash programmers to interact with any compliant ECU regardless of manufacturer.

2.1 ISO 14229 Multi-Part Structure

Part	Title	Transport / Focus
ISO 14229-1:2020	Application Layer (Core UDS specification)	All transports — defines all 27+ services
ISO 14229-2	Session layer requirements	Protocol timing (P2, P2*, S3), session management
ISO 14229-3	UDSonCAN	ISO-TP (ISO 15765-2) requirements for UDS
ISO 14229-4	Requirements for CAN-based networks	CAN 2.0 / CAN FD specifics
ISO 14229-5	UDS on IP networks (UDSonIP)	DoIP specifics (ISO 13400)
ISO 14229-6	Requirements for K-Line networks	K-Line / KWP2000 transport
ISO 14229-7	Requirements for LIN	LIN diagnostics transport
ISO 14229-8	Requirements for Ethernet	Extended Ethernet / CAN-FD requirements

2.2 Message Structure

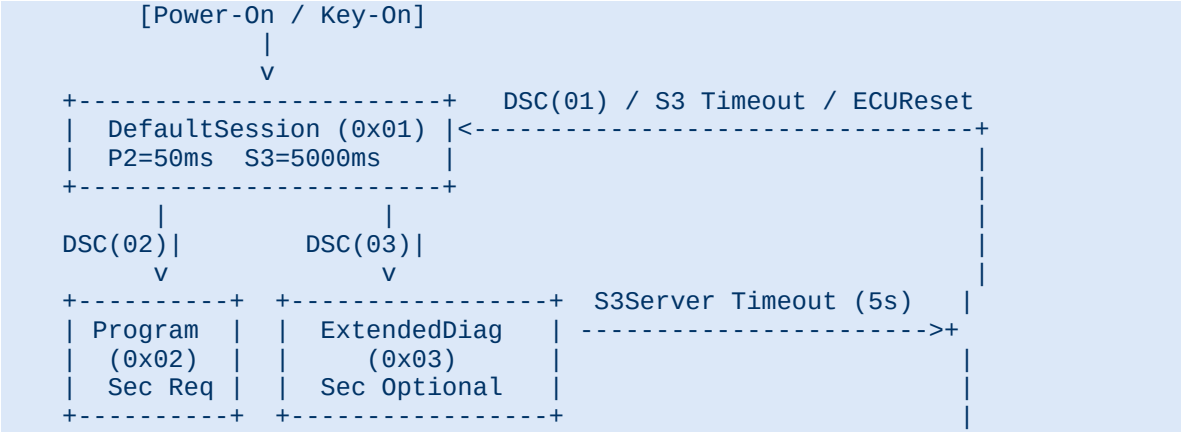
```
// UDS Request Frame Structure
+---------+-------------+-------------------------+
| SID     | SubFunction  | Data Parameters          |
| 1 byte  | 0 or 1 byte  | 0..4093 bytes            |
+---------+-------------+-------------------------+

// Positive Response: SID + 0x40 (response SID)
+-----------+-------------+-------------------------+
| SID+0x40  | SubFunction  | Response Data            |
| 1 byte    | 0 or 1 byte  | 0..4093 bytes            |
+-----------+-------------+-------------------------+

// Negative Response: Fixed 3-byte structure
+--------+-----+------+
| 0x7F   | SID | NRC  |
| 1 byte |1 by |1 by  |
+--------+-----+------+

// suppressPosRspMsgIndicationBit (SPRMIB) — bit 7 of SubFunction:
// Bit7 = 0: ECU MUST send positive response
// Bit7 = 1: ECU MUST NOT send positive response
// Example: [0x3E][0x80] = TesterPresent with suppressed response
```

2.3 Diagnostic Sessions

Session ID	Session Name	Access Level	Typical Use
0x01	defaultSession	Open / No auth required	DTC readout, VIN query, MIL status, OBD

Session ID	Session Name	Access Level	Typical Use
0x02	programmingSession	SecurityAccess required	ECU flashing, key learning, NvM write
0x03	extendedDiagnosticSession	SecurityAccess optional	ECU config, calibration, DID write, I/O
0x04	safetySystemDiagnosticSession	Highest security level	Access to safety-critical parameters
0x40-0x5F	vehicleManufacturerSpecific	OEM-defined	EOL session, production-line testing
0x60-0x7E	systemSupplierSpecific	Supplier-defined	Tier-1 supplier test sessions
0x60	vehicleOBDDiagnosticSession (WWH-OBD)	Open	Heavy-duty WWH-OBD diagnostics

Session state machine — key transitions:



2.4 UDS Timing Parameters (ISO 14229-2)

Parameter	Default Value	Description
P2_server	50 ms	Time from end of tester request to start of ECU response (max)
P2*_server	5000 ms	Extended time after 0x78 NRC (ResponsePending)
P2_client	150 ms	Tester wait time before timeout
P2*_client	5300 ms	Tester extended wait after receiving 0x78 NRC
S3_client	2000 ms	Tester must send TesterPresent (0x3E) within this interval
S3_server	5000 ms	ECU session timeout — returns to default if no request received
P3_client	100 ms	Min time between end of response and start of next request

Session Timeout Handling

If a non-default session is active and the ECU does not receive any request (including TesterPresent 0x3E 0x80) within S3_server (5000 ms), the ECU MUST silently return to the default session. Re-enabling normal communication (0x28) and other rollback actions are also performed. The tester must send 0x3E 0x80 (suppressPosRsp set) every 1500 ms to keep the session alive without bus traffic.

2.5 Security Access (0x27) — Seed & Key

```
// Security Access Protocol Flow

STEP 1: Request Seed
  Tester --> ECU: [0x27][0x01]  // subFunction 0x01 = requestSeed level 1
  ECU --> Tester: [0x67][0x01][SEED_B3][SEED_B2][SEED_B1][SEED_B0]
```

```
STEP 2: Compute Key (OEM algorithm, simplified XOR example)
  uint32_t seed = (S0<<24)|(S1<<16)|(S2<<8)|S3;
  uint32_t key  = (seed ^ 0xA5A5A5A5) + 0x12345678;  // OEM-specific

STEP 3: Send Key
  Tester --> ECU: [0x27][0x02][KEY_B3][KEY_B2][KEY_B1][KEY_B0]
  IF key matches: ECU --> Tester: [0x67][0x02]  // Access Granted
  ELSE:           ECU --> Tester: [0x7F][0x27][0x35]  // NRC: invalidKey

// SubFunction mapping: Odd = requestSeed, Even = sendKey
// 0x01/0x02 = Level 1 (programming), 0x03/0x04 = Level 2 (calibration)

// If already unlocked (ECU returns zero seed):
  ECU --> Tester: [0x67][0x01][0x00][0x00][0x00][0x00]  // Already unlocked
```

Security Access — ISO 21434 Recommendation

ISO 14229-1:2020 allows up to 126 security levels (odd = requestSeed, even = sendKey). For safety-critical ECUs, ISO 21434 recommends HSM-backed seed generation (Aurix HSM, SHE) and certificate-based Service 0x29 Authentication for programming-level access. Lockout after max failed attempts (NRC 0x36). Mandatory time delay (NRC 0x37). Physical OBD port access control for remote attack surfaces.

2.6 Addressing Modes

Mode	Description	Usage
Physical Addressing	1-to-1 unicast: tester to one specific ECU	Read/write DID, flash programming, RID
Functional Addressing	1-to-N broadcast: all matching ECUs respond	TesterPresent, ClearDTC, DTCInformation
Remote Diagnostic	Via sub-network router/gateway (ISO 14229-4)	Accessing ECUs behind central gateway

2.7 Negative Response Codes (NRC) — Complete Table

NRCs are returned in the third byte of a negative response frame [0x7F][SID][NRC]:

NRC Code	Mnemonic	Description
0x10	generalReject	Request not accepted; unspecified reason
0x11	serviceNotSupported	SID not recognized or not supported by ECU
0x12	subFunctionNotSupported	Sub-function value not supported
0x13	incorrectMessageLengthOrInvalidFormat	Wrong number of bytes or invalid encoding
0x14	responseTooLong	Response data exceeds transport layer capacity
0x21	busyRepeatRequest	ECU temporarily busy; tester should retry after delay
0x22	conditionsNotCorrect	Current ECU state does not permit execution
0x24	requestSequenceError	Request out of expected order (e.g., SendKey before RequestSeed)
0x25	noResponseFromSubnetComponent	Gateway received request but sub-ECU did not respond
0x26	failurePreventsExecution	An active fault prevents the requested action
0x31	requestOutOfRange	DID/RID/address value not supported or out of valid range
0x33	securityAccessDenied	Service requires security unlock not yet performed

NRC Code	Mnemonic	Description
0x34	authenticationRequired	Authentication (0x29) is required (ISO 14229-1:2020)
0x35	invalidKey	Key in SecurityAccess request does not match expected
0x36	exceededNumberOfAttempts	Max failed security access attempts reached; lockout active
0x37	requiredTimeDelayNotExpired	Security access delay timer still running
0x38	uploadDownloadNotAccepted	Transfer request rejected (invalid address, memory busy)
0x39	transferDataSuspended	Transfer data suspended; active fault or memory protection prevented data transfer from completing
0x3A	oemProgrammingFailure (OEM-specific 0x70)	Flash write or erase operation failed; memory cell error, sector protection, or voltage fault (ISO 14229-1 standard code)
0x3B	oemWrongBlockSeq (OEM-specific 0x73)	Block sequence counter in TransferData (0x36) does not match expected value; indicates lost or out-of-order block (ISO 14229-1 standard code)
0x70	generalProgrammingFailure	OEM-specific range; some manufacturers use 0x70 for flash write/erase failure (standard code: 0x3A generalProgrammingFailure)
0x72	oemProgrammingFailure2 (OEM-specific 0x72)	OEM-specific range; some manufacturers use 0x72 for erroneous memory block errors (standard code: 0x3A generalProgrammingFailure)
0x73	wrongBlockSequenceCounter	OEM-specific range; some manufacturers use 0x73 for block sequence errors (ISO 14229-1 standard code is 0x3B wrongBlockSequenceCounter)
0x78	requestCorrectlyReceivedResponsePending	ECU needs more time; tester must restart P2* timeout
0x7E	subFunctionNotSupportedInActiveSession	Sub-function valid but not in current session
0x7F	serviceNotSupportedInActiveSession	Service valid but not available in current session
0x88	vehicleSpeedTooHigh	Safety interlock — vehicle speed outside limit
0x89	engineRunning	ECU requires engine off for this operation

CHAPTER 3

Service Identifiers (SIDs) — Complete Reference

ISO 14229-1:2020 defines 27+ standardized services, each identified by a Service ID (SID). The positive response SID is always the request SID OR-ed with 0x40. Services are grouped into functional categories.

3.1 Complete SID Overview Table

SID (Hex)	Service Name	Session	Sec Req	Category
0x10	DiagnosticSessionControl	D/E/P	No	Session Control
0x11	ECUReset	D/E/P	Optional	Session Control
0x14	ClearDiagnosticInformation	D/E/P	Optional	Stored Data
0x19	ReadDTCInformation	D/E/P	No	Stored Data
0x22	ReadDataByIdentifier (RDBI)	D/E/P	Optional	Data Transmission
0x23	ReadMemoryByAddress	E/P	Yes	Data Transmission
0x24	ReadScalingDataByIdentifier	E/P	No	Data Transmission
0x27	SecurityAccess	E/P	N/A	Security
0x28	CommunicationControl	E/P	Optional	Comm Control
0x29	Authentication (NEW 2020)	D/E/P	N/A	Security
0x2A	ReadDataByPeriodicIdentifier	E	Optional	Data Transmission
0x2C	DynamicallyDefineDataIdentifier	E/P	Optional	Data Transmission
0x2E	WriteDataByIdentifier (WDBI)	E/P	Optional	Data Transmission
0x2F	InputOutputControlByIdentifier	E	Yes	I/O Control
0x31	RoutineControl	D/E/P	Optional	Remote Activation
0x34	RequestDownload	P	Yes	Upload/Download
0x35	RequestUpload	P	Yes	Upload/Download
0x36	TransferData	P	Yes	Upload/Download
0x37	RequestTransferExit	P	Yes	Upload/Download
0x38	RequestFileTransfer	P	Yes	Upload/Download
0x3D	WriteMemoryByAddress	E/P	Yes	Data Transmission
0x3E	TesterPresent	D/E/P	No	Comm Control
0x83	AccessTimingParameter	E/P	Optional	Comm Control
0x84	SecuredDataTransmission	D/E/P	N/A	Comm Control
0x85	ControlDTCSetting	E/P	Optional	Stored Data
0x86	ResponseOnEvent	D/E	Optional	Comm Control
0x87	LinkControl	E/P	Optional	Comm Control

Legend: D = Default Session (0x01), E = Extended Diagnostic Session (0x03), P = Programming Session (0x02)

3.2 suppressPosRspMsgIndicationBit (SPRMIB)

Bit 7 of the sub-function byte is the SPRMIB. When set, the ECU suppresses the positive response. Widely used for TesterPresent to maintain sessions without generating bus traffic:

```
/* TesterPresent with SUPPRESSED response (bus-silent session keep-alive) */
```

```
Tester TX: [0x3E][0x80]  -- SID=3E, SubFunc=0x80 (0x00 | SPRMIB)
ECU:       <no response> -- Positive response suppressed, S3 timer reset

/* TesterPresent with response (explicit keep-alive) */
Tester TX: [0x3E][0x00]  -- SubFunc=0x00
ECU    RX: [0x7E][0x00]  -- 0x3E+0x40=0x7E, sub-function echoed

/* Any valid service also resets S3 timer, e.g.: */
Tester TX: [0x22][0xF1][0x86]  -- ReadDataByIdentifier: active session
ECU    RX: [0x62][0xF1][0x86][0x03]  -- currentSession = Extended
```

3.3 Key Services — Detailed Reference

3.3.1 DiagnosticSessionControl (0x10)

```
Request:  [0x10][sessionType]
Response: [0x50][sessionType][P2_HI][P2_LO][P2X_HI][P2X_LO]

Example — Enter Extended Diagnostic Session:
  Request:  10 03
  Response: 50 03 00 32 01 F4
            // P2 = 0x0032 = 50ms, P2* = 0x01F4 * 10 = 5000ms

Example — Enter Programming Session:
  Request:  10 02
  Response: 50 02 00 19 01 F4
```

3.3.2 ECUReset (0x11)

```
Request:  [0x11][resetType]
Response: [0x51][resetType]

Reset Types:
  0x01 = hardReset        — Full power-cycle equivalent (re-init all NvM flush)
  0x02 = keyOffOnReset    — Simulate key-off / key-on cycle
  0x03 = softReset        — Software warm restart; preserves some RAM state
  0x04 = enableRapidPowerShutDown  (battery-powered ECUs)
  0x05 = disableRapidPowerShutDown
  0x40-0x5F = vehicleManufacturerSpecific (e.g., 0x41 = reset + clear adaptive)
```

3.3.3 ClearDiagnosticInformation (0x14)

```
Request:  [0x14][groupOfDTC_HI][groupOfDTC_MID][groupOfDTC_LO]
Response: [0x54]  (no additional data)

Special DTC Group Values:
  0xFF 0xFF 0xFF = allSupportedDTCs    (clear all)
  0xFF 0xFF 0xFE = allPendingDTCs
  0xFF 0xFF 0xFD = allMirrorMemoryDTCs

// AUTOSAR: Dcm calls Dem_ClearDTC() which clears status bits
// and removes DTC from primary/mirror/permanent memory.
// Permanent DTCs (OBD emission) require separate handling per WWH-OBD.
```

3.3.4 ReadDataByIdentifier (0x22) and WriteDataByIdentifier (0x2E)

```
/* Read VIN (DID 0xF190) */
Request:  22 F1 90
Response: 62 F1 90 4D 41 4B 31 4A 34 54 32 38 37 4B 50 31 31 32 33 34
          //          M  A  K  1  J  4  T  2  8  7  K  P  1  1  2  3  4
```

```
/* Read multiple DIDs in single request */
Request:  22 F1 90 F1 87  (VIN + Spare Part Number)
Response: 62 F1 90 <17 bytes VIN> F1 87 <data...>

/* Write VIN (End-of-Line, requires Prog session + Security) */
Request:  2E F1 90 4D 41 4B 31 4A 34 54 32 38 37 4B 50 31 31 32 33 34
Response: 6E F1 90  // positive — DID echoed
```

3.3.5 RoutineControl (0x31)

```
Request:  [0x31][subFunction][RID_HI][RID_LO][optionRecord...]
Response: [0x71][subFunction][RID_HI][RID_LO][routineStatusRecord...]

Sub-Functions: 0x01=startRoutine  0x02=stopRoutine  0x03=requestRoutineResults

Standard RIDs:
  0xFF00 = eraseMemory              (flash sector erase)
  0xFF01 = checkProgrammingDependencies
  0xFF02 = eraseMirrorMemoryDTCs

Example — Start flash erase routine:
  Request:  31 01 FF 00 [addr 4B] [length 4B]
  Response: 71 01 FF 00 01  // routineInfo: in progress
  Request:  31 03 FF 00
  Response: 71 03 FF 00 00  // routineStatus: completed OK
```

3.3.6 Flash Programming Sequence (0x34/0x36/0x37)

```
/* Complete Flash Programming Sequence */
1. 10 02  -->  50 02 ...         Enter Programming Session
2. 27 01  -->  67 01 [SEED]      Request Seed (Level 1)
3. 27 02 [KEY]  --> 67 02        Send Key (Access Granted)
4. 28 03 01  -->  68 03          Disable Rx+Tx comm
5. 85 02 FF FF FF --> C5 02      Disable DTC storage
6. 31 01 FF 00 [addr][size]      Erase memory (async)
   31 03 FF 00  --> 71 03 FF 00 00  Check erase done
7. 34 00 44 [memAddr 4B] [size 4B]  RequestDownload
   --> 74 40 [maxBlockLen 4B]        ECU confirms max block size
8. 36 01 [up to maxBlockLen bytes]   TransferData block 1
   --> 76 01                         Positive ack
   36 02 [...data...]  --> 76 02       Repeat for each block
9. 37  -->  77                       RequestTransferExit (CRC verify)
10. 31 01 FF 01  --> verify dependencies pass
11. 28 00 01  -->  68 00              Re-enable comm
12. 85 01 FF FF FF  -->  C5 01        Re-enable DTC setting
13. 11 01  -->  51 01                 ECUReset (hard reset)
```

3.4 CommunicationControl (0x28)

```
/* Disable normal Rx+Tx communication during programming */
Tester TX: [0x28][0x03][0x01]
           SID   disableRxAndTx  communicationType=normalCommunicationMessages
ECU    RX: [0x68][0x03]  -- positive response

/* Re-enable after programming */
Tester TX: [0x28][0x00][0x01]  -- enableRxAndTx
ECU    RX: [0x68][0x00]

Sub-functions: 0x00=enableRxAndTx, 0x01=enableRxDisableTx,
               0x02=disableRxEnableTx, 0x03=disableRxAndTx
```

3.5 ControlDTCSetting (0x85)

```
/* Disable all DTC storage (used during EOL actuator tests) */
Tester TX: [0x85][0x02][0xFF][0xFF][0xFF]
           SID   disableDTCSetting  groupOfDTC=allDTCs
ECU    RX: [0xC5][0x02]

/* Re-enable DTC setting */
Tester TX: [0x85][0x01][0xFF][0xFF][0xFF]
ECU    RX: [0xC5][0x01]
```

3.6 ResponseOnEvent (0x86)

Service 0x86 allows the ECU to asynchronously push response data to the tester when a configurable event occurs, without tester polling. Useful for real-time fault monitoring:

Sub-function	Value	Description
stopResponseOnEvent	0x00	Stop sending event-driven responses; deactivate all configured events
onDTCStatusChange	0x01	ECU pushes response when any DTC status byte changes value
onTimerInterrupt	0x02	ECU pushes response at a fixed configurable timer interval
onChangeOfDataIdentifier	0x03	ECU pushes response when specified DID value changes
reportActivatedEvents	0x04	Report all currently active event configurations to tester
startResponseOnEvent	0x05	Activate event-driven response using previously configured event
clearResponseOnEvent	0x06	Clear all event configurations from ECU memory

3.7 Authentication Service (0x29) — ISO 14229-1:2020

Service 0x29 (Authentication) was introduced in ISO 14229-1:2020 to replace basic seed-key with certificate-based or HMAC challenge-response authentication backed by HSM:

Sub-function	Value	Description
deAuthenticate	0x00	Remove all granted access levels
verifyCertificateUnidirectional	0x01	ECU verifies tester certificate (one-way)
verifyCertificateBidirectional	0x02	Mutual certificate verification (both directions)
proofOfOwnership	0x03	Challenge-response ownership proof after certificate exchange
transmitCertificate	0x04	Send certificate chain to ECU for validation
requestChallengeForAuthentication	0x05	Request nonce/challenge from ECU
verifyProofOfOwnershipUnidirectional	0x06	ECU verifies HMAC/signature from tester
authenticationConfiguration	0x07	Read authentication capabilities of ECU

CHAPTER 4

OBD-II Standards and Process Identifiers (PIDs)

OBD-II (On-Board Diagnostics II) is an emission-focused diagnostic standard mandated in the USA since 1996 (EPA), Europe since 2001 (EOBD), and most global markets thereafter. It defines a standardized interface for emission-related self-monitoring, fault reporting, and live data access via a standardized 16-pin Data Link Connector (DLC).

4.1 OBD-II DLC Pinout (SAE J1962)

The 16-pin OBD-II Data Link Connector (J1962) is mandated to be within 60 cm of the steering wheel, accessible without tools:

Pin	Signal	Standard	Notes
2	SAE J1850 Bus+	J1850 PWM/VPW	Legacy GM/Ford
4	Chassis Ground	All	Vehicle body ground reference
5	Signal Ground	All	ECU signal reference ground
6	CAN High (J-2284)	ISO 15765-4	Primary OBD-II CAN High line
7	ISO 9141-2 K-Line	ISO 9141-2	Legacy European K-Line
10	SAE J1850 Bus-	J1850 PWM	Differential minus
14	CAN Low (J-2284)	ISO 15765-4	Primary OBD-II CAN Low line
15	ISO 9141-2 L-Line	ISO 9141-2	Legacy initialization line
16	+12V Battery	All	Unswitched permanent power

4.2 OBD-II Standards Ecosystem

- SAE J1979-2:2022 — Updated OBD-II services for CAN-based vehicles (supersedes J1979 for CAN)
- ISO 15031-5:2015 — International equivalent covering OBD communication services
- SAE J1939-73:2022 — Heavy-duty OBD (HD-OBD) extensions for trucks and buses
- ISO 27145 (WWH-OBD) — World-Wide Harmonized OBD for global regulatory harmonization
- SAE J1962:2012 — 16-pin OBD-II DLC connector pinout and mechanical spec
- ISO 15765-4:2021 — CAN-based OBD-II transport; 250/500 kbps bit rates

4.3 OBD-II CAN Transport (ISO 15765-4)

OBD-II uses ISO 15765-2 (CAN TP) for segmentation. Tester uses 0x7DF (functional broadcast) or 0x7E0-0x7E7 (physical). ECU responds on 0x7E8-0x7EF (+8 offset):

```
/* Single Frame OBD Request: Mode 01, PID 0x0D (Vehicle Speed) */
Byte 0 (PCI+DL): 0x02  -- Single Frame, 2 data bytes
Byte 1 (SID):    0x01  -- Mode 01: Show Current Data
Byte 2 (PID):    0x0D  -- Vehicle Speed PID
Bytes 3-7:       0x00  -- Padding (ISO 15765-4 requires 8-byte frame)

/* ECU Positive Response (on 0x7E8) */
Byte 0: 0x04  -- 4 data bytes follow
Byte 1: 0x41  -- 0x01 + 0x40 = positive response to Mode 01
Byte 2: 0x0D  -- PID echoed
Byte 3: 0x3C  -- Value = 60 decimal => 60 km/h (formula: A = km/h)
```

4.4 CAN TP Multi-Frame Protocol (ISO 15765-2)

For OBD responses exceeding 7 bytes (e.g., VIN readout, multi-PID responses), ISO 15765-2 multi-frame protocol is used:

```
/* STEP 1: ECU sends First Frame (FF) — e.g., 20 bytes of data */
Byte 0: 0x10 | (DL_high)  -- PCI = 1 (FF), upper 4 bits of total length
Byte 1: DL_low             -- lower 8 bits of total data length (e.g. 0x14 = 20)
Bytes 2-7: first 6 data bytes of response

/* STEP 2: Tester sends Flow Control (FC) frame — allowing ECU to continue */
Byte 0: 0x30  -- PCI = 3 (FC), FlowStatus = 0 (ContinueToSend)
Byte 1: 0x00  -- BlockSize = 0 (no limit on consecutive frames)
Byte 2: 0x0A  -- STmin = 10 ms (minimum separation time between CFs)

/* STEP 3: ECU sends Consecutive Frames (CF) */
Byte 0: 0x21  -- PCI = 2 (CF), SequenceNumber = 1
Bytes 1-7: next 7 data bytes

Byte 0: 0x22  -- SequenceNumber = 2
Bytes 1-7: remaining data bytes (padded if necessary)

/* Sequence counter wraps: 0x21..0x2F, then 0x20, 0x21, ... */
```

4.5 OBD-II Diagnostic Services (Modes)

Mode	Service Name	Description
$01 (0x01)	Show current data	Read live sensor/calculated parameter values
$02 (0x02)	Show freeze frame data	Read sensor snapshot captured when DTC was set
$03 (0x03)	Show stored DTCs	Read all confirmed emission-related DTCs
$04 (0x04)	Clear DTCs and stored values	Clear all emission DTCs, freeze frames, readiness
$05 (0x05)	O2 sensor test results (non-CAN)	O2 sensor data (K-line / ISO 9141 only)
$06 (0x06)	On-board monitoring test results	Per-monitor OBDMID values and test min/max/actual
$07 (0x07)	Show pending DTCs	DTCs detected this drive cycle, not yet confirmed
$08 (0x08)	Control on-board systems	Activate/deactivate specific monitors or tests
$09 (0x09)	Request vehicle information	VIN, calibration ID, CVN, ECU name
$0A (0x0A)	Permanent / Cleared DTCs	DTCs that survive Mode 04; cannot be user-cleared

4.6 Mode $01 PIDs — Live Data (Selected)

PID	Parameter	Bytes	Formula / Units	Range
0x00	PIDs supported [01-20] bitmask	4	Bit mask	Bit field
0x01	Monitor status since DTCs cleared	4	Readiness bitmask	Bit field
0x04	Calculated engine load	1	A / 2.55 (%)	0-100%
0x05	Engine coolant temperature	1	A - 40 (degC)	-40 to 215 degC
0x06	Short-term fuel trim Bank 1	1	(A-128)*100/128 (%)	-100 to +99.2%
0x07	Long-term fuel trim Bank 1	1	(A-128)*100/128 (%)	-100 to +99.2%
0x0A	Fuel pressure gauge	1	3*A (kPa)	0-765 kPa

PID	Parameter	Bytes	Formula / Units	Range
0x0B	Intake manifold pressure	1	A (kPa)	0-255 kPa
0x0C	Engine RPM	2	(256A+B)/4 (rpm)	0-16383.75 rpm
0x0D	Vehicle speed	1	A (km/h)	0-255 km/h
0x0E	Timing advance	1	A/2 - 64 (deg BTDC)	-64 to 63.5 deg
0x0F	Intake air temperature	1	A - 40 (degC)	-40 to 215 degC
0x10	MAF air flow rate	2	(256A+B)/100 (g/s)	0-655.35 g/s
0x11	Throttle position	1	A/2.55 (%)	0-100%
0x1F	Run time since engine start	2	256A+B (s)	0-65535 s
0x21	Distance with MIL on	2	256A+B (km)	0-65535 km
0x2F	Fuel tank level	1	A/2.55 (%)	0-100%
0x30	Warm-ups since DTCs cleared	1	A (count)	0-255
0x31	Distance since DTCs cleared	2	256A+B (km)	0-65535 km
0x46	Ambient air temperature	1	A-40 (degC)	-40 to 215 degC
0x49	Accelerator pedal position D	1	A/2.55 (%)	0-100%
0x4D	Time run with MIL on	2	256A+B (min)	0-65535 min
0x4E	Time since DTCs cleared	2	256A+B (min)	0-65535 min
0x5C	Engine oil temperature	1	A-40 (degC)	-40 to 215 degC
0x5E	Engine fuel rate	2	0.05*(256A+B) (L/h)	0-3276.75 L/h
0x61	Demanded engine % torque	1	A-125 (%)	-125 to 130%
0x62	Actual engine % torque	1	A-125 (%)	-125 to 130%

4.7 Mode $09 — Vehicle Information PIDs

INFOTYPE	Parameter	Format
0x02	Vehicle Identification Number (VIN)	17 ASCII characters (ISO 3779)
0x04	Calibration ID (CAL ID)	ASCII, multiple 16-char blocks, one per ECU
0x06	Calibration Verification Number (CVN)	4 bytes per ECU (CRC-32 or OEM algo)
0x0A	ECU name	20 ASCII characters
0x0C	Engine Serial Number (ESN)	Variable length ASCII

4.8 OBD-II Readiness Monitors

Monitor	Full Name	Type	Applicability
MIS	Misfire monitor	Continuous	SI and CI engines
FUE	Fuel system monitor	Continuous	SI and CI engines
CCM	Comprehensive component monitor	Continuous	All vehicles
CAT	Catalyst (TWC) efficiency monitor	Non-continuous	SI engines
HO2S	Heated oxygen sensor monitor	Non-continuous	SI engines
EGR	EGR system monitor	Non-continuous	SI and CI engines
EVAP	Evaporative system monitor	Non-continuous	SI engines
PM	PM filter monitor	Non-continuous	CI engines with DPF
NCAT	NOx catalyst monitor	Non-continuous	CI engines with SCR/LNT

OBD-II vs UDS Coexistence

OBD-II and UDS can coexist in the same ECU. The ECU switches to OBD mode when addressed via the functional CAN ID 0x7DF and handles UDS when addressed via its physical UDS CAN ID (e.g., 0x760). AUTOSAR DCM handles both transparently through separate protocol configuration instances (DcmDslProtocol instances for OBD and UDS).

CHAPTER 5

Data Identifiers (DIDs) and Routine Identifiers (RIDs)

Data Identifiers (DIDs) are 2-byte identifiers used in UDS services 0x22 (ReadDataByIdentifier), 0x2E (WriteDataByIdentifier), 0x24 (ReadScalingData), 0x2C (DynamicDefine), and 0x2F (InputOutputControl) to reference specific data records. DIDs provide a structured namespace for all ECU data accessible via diagnostics.

5.1 DID Address Space Allocation

DID Range	Category	Description
0x0000-0x00FF	ISO Reserved	Reserved by ISO 14229; not for use
0x0100-0x01FF	ISO Assigned	OBD-related DIDs (periodical and on-request)
0x0200-0xEFFF	OEM / System Supplier	Vehicle manufacturer and supplier specific DIDs
0xF000-0xF0FF	Network Config DIDs	Network communication configuration data
0xF100-0xF1FF	Vehicle Manufacturer DIDs	Standard OEM-assigned DIDs for vehicle/ECU identification
0xF200-0xF2FF	Periodic DIDs	Used with ReadDataByPeriodicIdentifier (0x2A)
0xF300-0xF3FF	Dynamically Defined DIDs	Used with DynamicallyDefineDataIdentifier (0x2C)
0xF400-0xF4FF	Vehicle Manufacturer Defined	Additional OEM-specific range
0xF500-0xFCFF	System Supplier Defined	Tier-1 supplier specific DIDs
0xFD00-0xFEFF	Reserved	ISO 14229 reserved range
0xFF00-0xFFFF	ISO Assigned	ISO-assigned special-purpose DIDs

5.2 Standard Vehicle/ECU Identification DIDs (0xF1xx)

The F1xx range contains standardized DIDs that AUTOSAR-compliant ECUs are expected to implement. These are referenced in ISO 14229-1 Annex C:

DID	Name	Format	Description
0xF186	activeDiagnosticSession	1 byte	Current session type (0x01/0x02/0x03)
0xF187	vehicleManufacturerSparePartNumber	ASCII	Part number for warranty/procurement
0xF18A	systemSupplierIdentifier	3 bytes	DUNS number or supplier code
0xF18B	ecuManufacturingDate	3 bytes	BCD date: YYYYMMDD format
0xF18C	ecuSerialNumber	ASCII	Hardware serial number of ECU
0xF190	VIN (Vehicle Identification Number)	17 ASCII	ISO 3779 VIN - 17 alphanumeric
0xF191	vehicleManufacturerECUHardwareVersion	ASCII	Hardware version string
0xF192	systemSupplierECUHardwareNumber	ASCII	Supplier-assigned HW part number
0xF193	systemSupplierECUHardwareVersionNumber	ASCII	Supplier HW version
0xF194	systemSupplierECUSoftwareNumber	ASCII	Supplier-assigned SW part number

DID	Name	Format	Description
0xF195	systemSupplierECUSoftwareVersionNumber	ASCII	SW version string (e.g., "02.04.01")
0xF196	exhaustRegulationOrTypeApprovalNumber	ASCII	Emission certification number
0xF197	systemNameOrEngineType	ASCII	System/engine type identifier
0xF198	repairShopCodeOrTesterSerialNumber	ASCII	Last tester serial number used
0xF199	programmingDate	4 bytes	Date of last ECU programming (YYYYMMDD BCD)
0xF19D	ecuInstallationDate	4 bytes	Date of ECU installation in vehicle
0xF19E	ODXFile	ASCII	Diagnostic description file reference (ODX)
0xF1A0	statusIndicationValueReset	1 byte	Used for OBD/emission indication status

5.3 DID Implementation in AUTOSAR

In AUTOSAR, DIDs are configured in the DCM module through DcmDsp (Diagnostic Service Processing) configuration containers. Each DID maps to one or more DcmDspData entries:

```
// AUTOSAR DCM DID Configuration (pseudocode representation)

DcmDspDid_F190:  // VIN DID
  DcmDspDidIdentifier     = 0xF190
  DcmDspDidDataByteOffset = 0
  DcmDspDidUsed           = TRUE
  DcmDspDidRef --> DcmDspData_VIN_Data

DcmDspData_VIN_Data:
  DcmDspDataType          = UINT8_N
  DcmDspDataSize          = 17 bytes
  DcmDspDataConditionCheckReadFnc  = VIN_ConditionCheckRead
  DcmDspDataReadDataFnc   = VIN_ReadData
  DcmDspDataWriteDataFnc  = VIN_WriteData
  DcmDspDataUsePort       = USE_DATA_SYNCH_FNC

// SWC callback functions:
Std_ReturnType VIN_ReadData(
    uint8* Data) {    // Output: 17-byte VIN
  Nvm_ReadBlock(NvM_BlockId_VIN, Data);
  return E_OK;
}

Std_ReturnType VIN_WriteData(
    const uint8* Data,
    Dcm_OpStatusType OpStatus,
    Dcm_NegativeResponseCodeType* ErrorCode) {
  if (!IsValidVIN(Data)) {
    *ErrorCode = DCM_E_REQUESTOUTOFRANGE;
    return E_NOT_OK;
  }
  Nvm_WriteBlock(NvM_BlockId_VIN, Data);
  return E_OK;
}
```

5.4 DynamicallyDefineDataIdentifier (0x2C)

Service 0x2C allows testers to compose custom DIDs at runtime by combining bytes from existing DIDs or memory addresses. This avoids pre-configuring every possible tester data request:

```
Request: [0x2C][subFunction][DID_HI][DID_LO][sourceDataRecords...]

Sub-Functions:
  0x01 = defineByIdentifier
  0x02 = defineByMemoryAddress
  0x03 = clearDynamicallyDefinedDataIdentifier

Example - Define dynamic DID 0xF301 from two existing DIDs:
  2C 01 F3 01   // define DID 0xF301
  F1 90 01 11   // sourceDataRecord 1: DID=0xF190, pos=1, size=17 (VIN)
  00 05 01 01   // sourceDataRecord 2: DID=0x0005, pos=1, size=1

// Response: 6C 01 F3 01

// Now reading DID 0xF301 returns concatenated data from F190 + 0005
  22 F3 01
  Response: 62 F3 01 <VIN data> <0x0005 data>
```

5.5 Routine Identifiers (RIDs)

Routine Identifiers are 2-byte identifiers used with RoutineControl (0x31) to invoke sequences or algorithms on the ECU. Unlike DIDs (passive data), RIDs represent executable routines.

5.5.1 Standard RID Range

RID Range	Owner	Examples
0x0000-0x00FF	ISO Reserved	Not for use
0x0100-0x01FF	ISO Assigned	`0x0200 = DeployLoopRoutineID, 0x0203 = EraseMirrorMemoryDTCs`
0x0200-0x02FF	OBD-related	`0x0202 = FlashProgrammingSuccess, 0x0206 = ValidateFlash`
0xE000-0xEFFF	System Supplier Specific	Supplier-defined test routines
0xF000-0xFF00	OEM Specific	Checksum, flash erase, variant coding, key learning
0xFF00	eraseMemory (standard)	Erase flash memory segment
0xFF01	checkProgrammingDependencies	Verify SW compatibility post-flash
0xFF02	eraseMirrorMemoryDTCs	Clear DEM mirror memory
0xFF03	checkProgrammingPreconditions	Verify vehicle state before flash

5.5.2 RID Implementation Pseudocode

```
// RoutineControl Handler Example: ECU Checksum Verification Routine
// RID = 0xFF01, startRoutine then requestRoutineResults

ROUTINE ChecksumVerify_StartRoutine(
    routineControlOptionRecord[],
    Dcm_NegativeResponseCodeType* ErrorCode):

  // Validate preconditions
  IF (engineState != ENGINE_OFF):
    *ErrorCode = DCM_E_CONDITIONSNOTCORRECT
    RETURN E_NOT_OK

  // Start async checksum computation
  ChecksumTask_Start(FLASH_START_ADDR, FLASH_END_ADDR)
  g_ChecksumStatus = CHECKSUM_IN_PROGRESS
  RETURN E_OK  // DCM sends positive response
```

```
ROUTINE ChecksumVerify_RequestResults(
    routineStatusRecord[],
    uint8* routineStatusRecordLength,
    Dcm_NegativeResponseCodeType* ErrorCode):

  IF (g_ChecksumStatus == CHECKSUM_IN_PROGRESS):
    routineStatusRecord[0] = 0x01  // still running
    routineStatusRecord[1] = 0x00  // no result yet
    *routineStatusRecordLength = 2
    RETURN E_OK

  ELSE IF (g_ChecksumStatus == CHECKSUM_PASSED):
    routineStatusRecord[0] = 0x02  // completed
    routineStatusRecord[1] = 0x00  // result: PASS
    RETURN E_OK

  ELSE: // CHECKSUM_FAILED
    routineStatusRecord[0] = 0x02  // completed
    routineStatusRecord[1] = 0x01  // result: FAIL
    RETURN E_OK
```

CHAPTER 6

Diagnostic Trouble Codes (DTCs)

Diagnostic Trouble Codes (DTCs) are standardized fault codes that an ECU stores when it detects a monitored component or system operating outside its normal range. DTCs are the primary communication mechanism between vehicle diagnostics and technicians. ISO 14229-1 and SAE J2012 define the DTC format, status, and lifecycle.

6.1 DTC Format and Structure

A UDS DTC consists of three bytes according to ISO 14229-1:

```
// ISO 14229-1 DTC Format (3 bytes)
+----------+----------+----------+
| Byte 1   | Byte 2   | Byte 3   |
| High Byte| Mid Byte |Fail Type |
+----------+----------+----------+

// Byte 1 bit encoding (ISO 14229 DTC numbering):
  Bit 7-6: System category
    00 = Powertrain (P)
    01 = Chassis (C)
    10 = Body (B)
    11 = Network/UUDT (U)
  Bit 5-4: P0/P1/P2/P3 sub-category
    00 = Generic (SAE J2012 defined)
    01 = Manufacturer-specific
    10 = Generic (SAE J2012 defined)
    11 = Mixed
  Bit 3-0: System area (subsystem identifier)

// Examples:
  P0300 = 0x02 0x03 0x00   // Random/multiple cylinder misfire
  P0171 = 0x01 0x71 0x00   // Fuel system too lean (Bank 1)
  U0100 = 0x50 0x01 0x00   // Lost communication with ECM/PCM "A"
  B1050 = 0x91 0x50 0x00   // Manufacturer body DTC
```

6.2 DTC Status Byte - Bit Definitions

The DTC status byte (per ISO 14229-1 Annex D) is an 8-bit field where each bit represents a specific aspect of the DTC lifecycle. The ECU reports this byte in ReadDTCInformation responses:

Bit	Status Bit Name	Abbreviation	Description
0	testFailed	TF	Set when the associated monitor currently reports FAILED. Cleared when monitor reports PASSED.
1	testFailedThisOperationCycle	TFTOC	Set when monitor reported FAILED at any point in the current operation cycle. Never cleared within the operation cycle.
2	pendingDTC	PDTC	Set when testFailed first time in an operation cycle. Cleared if no failure in next complete operation cycle.
3	confirmedDTC	CDTC	Set after DTC confirmed per confirmation threshold. Once set, requires aging or explicit clearing to reset.
4	testNotCompletedSinceLastClear	TNCTSLC	Set after DTC clear. Cleared when monitor completes test (pass or fail).
5	testFailedSinceLastClear	TFSLC	Set on any FAILED event since last DTC clear. Never auto-

Bit	Status Bit Name	Abbreviation	Description
			cleared except via Mode 04 / 0x14.
6	testNotCompletedThisOperationCycle	TNCTOC	Set at operation cycle start. Cleared when test completes (pass or fail).
7	warningIndicatorRequested	WIR	Set when MIL or other indicator lamp is requested active. Drives Dem_GetIndicatorStatus() output.

```
// DTC Status Byte Example: 0x2F = 0010 1111
  Bit 0 (TF)      = 1: Test currently FAILING
  Bit 1 (TFTOC)   = 1: Failed this operation cycle
  Bit 2 (PDTC)    = 1: Pending DTC active
  Bit 3 (CDTC)    = 1: DTC is confirmed
  Bit 4 (TNCTSLC) = 0: Test completed since last clear
  Bit 5 (TFSLC)   = 1: Failed since last DTC clear
  Bit 6 (TNCTOC)  = 0: Test completed this operation cycle
  Bit 7 (WIR)     = 0: Warning indicator not requested
```

6.3 DTC Severity Classification

Severity Byte	Category	Description
0x20	maintenanceOnly	Requires maintenance but not immediately safety-relevant
0x40	checkAtNextHalt	Check vehicle at next opportunity, possible emission issue
0x80	checkImmediately	Immediate safety concern; vehicle should be checked now
0x00	noSeverity	DTC has no severity classification

6.4 ReadDTCInformation (0x19) - Sub-Functions

Service 0x19 is one of the most comprehensive UDS services, providing access to all DTC-related information through 22+ sub-functions:

SubFunc	Name	Description
0x01	reportNumberOfDTCByStatusMask	Count of DTCs matching a status mask filter
0x02	reportDTCByStatusMask	List of DTC+status pairs matching status mask
0x03	reportDTCSnapshotIdentification	List of DTCSnapshot (freeze frame) records stored
0x04	reportDTCSnapshotRecordByDTCNumber	Read freeze frame data for a specific DTC
0x05	reportDTCSnapshotRecordByRecordNumber	Read freeze frame by record number
0x06	reportDTCExtendedDataRecordByDTCNumber	Extended data (occurrence count, aging counter, etc.)
0x07	reportNumberOfDTCBySeverityMaskRecord	Count by severity
0x08	reportDTCBySeverityMaskRecord	List by severity and status
0x09	reportSeverityInformationOfDTC	Severity byte for a specific DTC
0x0A	reportSupportedDTCs	All supported DTCs regardless of status
0x0B	reportFirstTestFailedDTC	Chronologically first DTC that failed since clear
0x0C	reportFirstConfirmedDTC	First confirmed DTC since last clear
0x0D	reportMostRecentTestFailedDTC	Most recently failed DTC

SubFunc	Name	Description
0x0E	reportMostRecentConfirmedDTC	Most recently confirmed DTC
0x0F	reportMirrorMemoryDTCByStatusMask	DTCs from mirror memory
0x13	reportEmissionsOBDDTCByStatusMask	Emission-related DTCs only
0x14	reportDTCFaultDetectionCounter	Fault Detection Counter (-128..+127) per event
0x15	reportDTCWithPermanentStatus	WWH-OBD permanent DTCs (survive clear)
0x17	reportUserDefMemoryDTCByStatusMask	User-defined memory DTCs (AUTOSAR R21-11+)
0x55	reportWWHOBDDTCByMaskRecord	WWH-OBD DTC reporting with OBD relevance

6.5 ReadDTCInformation Examples

```
// Example 1: Read all confirmed+pending DTCs (0x02, statusMask=0x0C)
  Request:  19 02 0C
  // statusMask: 0x0C = bits 2 (PDTC) and 3 (CDTC)
  Response: 59 02 FF 00 00 00  // dtcStatusAvailabilityMask=0xFF
            P0300 27            // DTC + status byte
            P0171 2F            // DTC + status byte
  // Decoded: 02 03 00 27 = P0300, status 0x27
  //          01 71 00 2F = P0171, status 0x2F

// Example 2: Read fault detection counter (0x14)
  Request:  19 14
  Response: 59 14
            02 03 00 78    // DTC P0300, FDC=0x78 (positive = trending toward fail)
            01 71 00 00    // DTC P0171, FDC=0x00 (neutral)

// FDC range: -128 (passed 100%) to +127 (failed 100%)
// FDC > 0: fault detection trending toward confirmed
// FDC = 127: DTC is test-failed

// Example 3: Read snapshot (freeze frame) for DTC P0300
  Request:  19 04 02 03 00 FF  // DTC=P0300, recordNum=0xFF (all records)
  Response: 59 04 02 03 00
            00 F1 90 ...        // recordNumber=0, DID=0xF190 (VIN)
            01 00 0C ...        // recordNumber=1, DID=0x000C (RPM)
```

6.6 Freeze Frame (DTC Snapshot) Data

When a DTC is first confirmed, the ECU captures a snapshot of key operating parameters. This freeze frame is critical for fault diagnosis:

Freeze Frame DID	Parameter	Typical Content
0x000C	Engine RPM	RPM at time of fault confirmation
0x000D	Vehicle Speed	Speed in km/h
0x0005	Engine Coolant Temp	ECT in degC
0x0011	Throttle Position	TPS in %
0x0004	Engine Load	Calculated load in %
0x000B	Intake Manifold Pressure	MAP in kPa
0x0010	MAF Air Flow	MAF in g/s

Freeze Frame DID	Parameter	Typical Content
0x0007	Long-term Fuel Trim B1	LTFT in %
0xF186	Active Diagnostic Session	Session type at time of fault
Manufacturer DID	Operation Cycle Counter	Cycle count at confirmation

6.7 DTC Extended Data Records

Extended data records (sub-function 0x06) provide additional diagnostic context beyond the freeze frame snapshot:

ExtDataRecordNum	Content	Description
0x01	Occurrence Counter	Number of times DTC was confirmed (0-255)
0x02	Aging Counter	Healing cycles since last confirmed failure (0-255)
0x03	Aging Cycle Counter	Total aging cycles since last confirmation
0x04	Ovflind (Overflow Indicator)	Indicates if event memory overflowed
0x06	Current FDC	Current fault detection counter value
0x10-0x4F	OBD Supplement	OBD-required extended data (cycle count, warm-ups)
0x90-0xEF	OEM Specific	Customer-defined extended data records
0xFE	WWH-OBD ExtData	WWH-OBD specific data
0xFF	All Records	Special value: request all defined records

CHAPTER 7

Diagnostic Communication Manager (DCM)

The Diagnostic Communication Manager (DCM) is an AUTOSAR Basic Software (BSW) module defined in AUTOSAR_SWS_DiagnosticCommunicationManager. It implements the UDS and OBD-II application layer protocol, managing all diagnostic service requests received via the vehicle's communication bus and dispatching them to the appropriate service handlers.

7.1 DCM Architecture - Three Sub-Layers

DCM is internally structured into three functional sub-layers:

```
+-----------------------------------------------------------+
|              DCM - Diagnostic Communication Manager       |
+------------------+------------------+--------------------+
|  DSL             |  DSD             |  DSP               |
| Diagnostic       | Diagnostic       | Diagnostic         |
| Session Layer    | Service Dispatch | Service Processor  |
+------------------+------------------+--------------------+
| - Session mgmt   | - SID routing    | - Service handlers |
| - Security mgmt  | - SubFunc route  | - DID read/write   |
| - Timing (P2/S3) | - Session/Sec    | - RID execution    |
| - Addressing     |   validation     | - DTC access       |
| - Pdu Buffering  | - NRC generation | - Security Access  |
+------------------+------------------+--------------------+
          |                 |                   |
          v                 v                   v
+-----------------------------------------------------------+
|           PduR (PDU Router) -> CanTp / DoIP               |
+-----------------------------------------------------------+
```

7.2 DSL - Diagnostic Session Layer

The DSL manages the diagnostic session lifecycle, security state machine, and all protocol timing parameters.

7.2.1 Session State Machine

```
// DCM Session State Machine

            +---------------------------+
   INIT -->| DEFAULT_SESSION (0x01)    |
           | P2=50ms  S3=5000ms        |<--+ (on S3 timeout from any session)
           +----------+----------------+   |
                      |                    |
           [0x10 0x03] |        [0x10 0x02] |  [0x11 any or
                       |                    |   S3 timeout]
                       v                    |
           +---------------------------+    |
           | EXTENDED_DIAG_SESSION     |    |
           | (0x03)                    |    |
           | P2=50ms  S3=5000ms        +---+
           | Security optional         |
           +----------+----------------+
                      |
           [0x10 0x02] |  (require security)
                       v
           +---------------------------+
           | PROGRAMMING_SESSION (0x02)|
```

```
            | P2=50ms  S3=5000ms        |
            | Security required         |
            | ComControl: no normal msg |
            +---------------------------+
```

7.2.2 Security State Machine

```
// DCM Security Access State Machine

  +------------------+
  | LOCKED           |<----+ (on ECUReset, session change, S3 timeout)
  | (level = 0)      |     |
  +--------+---------+     |
           |               |
  [Rx 0x27 0x01]           |
  (requestSeed)            |
           v               |
  +------------------+     |
  | SEED_REQUESTED   |     |
  | GenerateSeed()   |     |
  | Tx: 0x67 0x01    |     |
  +--------+---------+     |
           |               |
  [Rx 0x27 0x02]           |
  (sendKey)                |
           v               |
  [Key == Expected?]       |
      /                   |
    YES           NO       |
     |            |        |
     v            v        |
  UNLOCKED    attemptCnt++ |
  [level=1]  IF cnt>=max:  |
     |       LOCKED+timer--+
  [Tx 0x67]  Tx 0x7F 0x35
```

7.3 DSP - Diagnostic Service Processor

The DSP implements individual service handlers. Each handler is registered in the DCM configuration and called by the DSD when a matching SID is received.

7.3.1 DCM Main Processing Loop (Pseudocode)

```
// AUTOSAR DCM MainFunction - called every Dcm_MainFunctionPeriod (e.g., 5ms)

FUNCTION Dcm_MainFunction():

  // 1. Session timer management
  IF (currentSession != DEFAULT_SESSION):
    s3Timer += mainFunctionPeriod
    IF (s3Timer >= S3_SERVER_TIMEOUT):
      Dcm_DslInternal_SessionTimerExpired()
      // -> reset to default session, reenable comm, clear security
      RETURN

  // 2. P2/P2* response timer
  IF (diagnosticRequestPending):
    p2Timer += mainFunctionPeriod
    IF (p2Timer >= P2_SERVER AND responseNotSentYet):
      Dcm_DspInternal_SendResponsePending()  // send 0x7F SID 0x78

  // 3. Process pending diagnostic request
```

```
  IF (newRequestReady):
    Dcm_DspInternal_ProcessRequest()

  // 4. Handle async service results
  IF (asyncServiceResultReady):
    Dcm_DspInternal_FinishAsyncService()


FUNCTION Dcm_DspInternal_ProcessRequest():

  serviceId = rxBuffer[0]

  // DSD: Find service configuration
  serviceConfig = Dcm_FindService(serviceId)
  IF (serviceConfig == NULL):
    Dcm_SendNRC(serviceId, DCM_E_SERVICENOTSUPPORTED)  // 0x11
    RETURN

  // DSD: Check session permission
  IF NOT Dcm_CheckSessionPermission(serviceConfig.allowedSessions):
    Dcm_SendNRC(serviceId, DCM_E_SERVICENOTSUPPORTEDINACTIVESESSION)  // 0x7F
    RETURN

  // DSD: Check security level
  IF NOT Dcm_CheckSecurityPermission(serviceConfig.requiredSecurityLevel):
    Dcm_SendNRC(serviceId, DCM_E_SECURITYACCESSDENIED)  // 0x33
    RETURN

  // DSP: Execute service handler
  result = serviceConfig.handler(rxBuffer, rxLen, txBuffer, &txLen)

  IF (result == E_OK):
    Dcm_SendPositiveResponse(txBuffer, txLen)
  ELSE IF (result == DCM_E_PENDING):
    // Async: handler will complete later, 0x78 sent by P2* timer
    diagnosticRequestPending = TRUE
  ELSE:
    Dcm_SendNRC(serviceId, errorCode)
```

7.3.2 ReadDataByIdentifier Handler (Pseudocode)

```
FUNCTION Dcm_0x22_ReadDataByIdentifier(
    uint8 *pMsgContext_reqData,
    uint16 reqDataLen,
    uint8 *pMsgContext_resData,
    uint16 *resDataLen):

  offset = 0
  resLen = 0

  // Process each DID in request (each DID = 2 bytes)
  WHILE (offset < reqDataLen):
    didHi  = pMsgContext_reqData[offset++]
    didLo  = pMsgContext_reqData[offset++]
    didId  = (didHi << 8) | didLo

    didConfig = Dcm_FindDid(didId)
    IF (didConfig == NULL):
      *ErrorCode = DCM_E_REQUESTOUTOFRANGE  // 0x31
      RETURN E_NOT_OK

    // Check session/security for this specific DID
```

```
    IF NOT Dcm_CheckDidPermissions(didConfig):
      *ErrorCode = DCM_E_SECURITYACCESSDENIED  // 0x33
      RETURN E_NOT_OK

    // Call DID read callback (SWC or NvM)
    dataPtr = &pMsgContext_resData[resLen + 2]
    result = didConfig->ReadDataFnc(dataPtr)
    IF (result != E_OK):
      *ErrorCode = DCM_E_CONDITIONSNOTCORRECT  // 0x22
      RETURN E_NOT_OK

    // Add DID echo to response
    pMsgContext_resData[resLen++] = didHi
    pMsgContext_resData[resLen++] = didLo
    resLen += didConfig->dataSize

  *resDataLen = resLen
  RETURN E_OK
```

7.4 DCM AUTOSAR Configuration Structure

DCM is configured via AUTOSAR ECUC parameters. Key configuration containers:

Container	Key Parameters	Purpose
DcmDsp	DcmDspDid, DcmDspRoutine, DcmDspSecurity	Core service processor config
DcmDspDid	DcmDspDidIdentifier, DcmDspDidDataRef	Each DID definition
DcmDspData	ReadDataFnc, WriteDataFnc, DataSize, UsePort	DID data accessor functions
DcmDspRoutine	DcmDspRoutineIdentifier, StartFnc, StopFnc, RequestResultsFnc	RID handler functions
DcmDspSecurity	SecurityLevel, GetSeedFnc, CompareKeyFnc	Security level config
DcmDspSession	SessionLevel, SessionForBoot, P2ServerMax	Session timing and access
DcmDsl	DcmDslProtocol, P2ServerMax, P2StarServerMax, S3Server	Transport/session timing
DcmDslProtocol	ProtocolType (UDS/OBD), RequestRef, ResponseRef	PDU routing for request/response
DcmDslBuffer	DcmDslBufferSize	Rx/Tx buffer sizes (max 4095 bytes typically)

7.5 DCM Transport Layer Integration (CAN/DoIP)

```
// Diagnostic PDU flow: CAN Frame -> CanTp -> PduR -> DCM

// On CAN: Physical addressing
//   Tester TX ID: 0x7E0 (physical request to ECU at 0x760)
//   ECU TX ID:    0x7E8 (physical response from ECU)

// Functional (broadcast) address:
//   0x7DF -> all OBD ECUs respond
//   0x7DF -> addressed functionally via mask 0x7FF

// ISO-TP single frame for 8-byte UDS request (Classical CAN 8 DLC):
//   CAN DLC = 8, Data = [0x05][SID][D0][D1][D2][D3][0x00][0x00]
//   CanTp N_PDU = 0x05 0x22 0xF1 0x90 0x00 0x00 ...  (read VIN)
```

```
// DoIP (Ethernet) header for UDS:
// | Proto Ver | Inv Ver | Payload Type | Payload Len |
// | 0x02 0xFD | 0xFD 0x02 | 0x80 0x01 | 0x00 0x00 0x00 0x09 |
// | Source addr(2) | Target addr(2) | UDS data...

// AUTOSAR callback sequence:
// CanTp_RxIndication() -> PduR_CanTpRxIndication() ->
// Dcm_StartOfReception() -> Dcm_CopyRxData() -> Dcm_TpRxIndication()
```

CHAPTER 8

Bootloader Diagnostics and Secure Boot

ECU bootloader software is the foundational layer that runs immediately after power-on and enables secure firmware programming, integrity verification, and safe application start-up. In automotive systems the bootloader and application SW reside in separate flash regions, and UDS Programming Session services are handled exclusively by the bootloader. ISO 14229-1, AUTOSAR BSW specifications, and OEM boot-security requirements govern the boot sequence, jump logic, and cryptographic validation.

8.1 Boot Software Architecture

An automotive ECU typically uses a two-stage boot structure: a Primary Boot Loader (PBL) in protected flash and a Secondary Boot Loader (SBL) in reprogrammable flash. The PBL is write-protected and contains the core flash driver and crypto verification routines; the SBL implements the full UDS programming stack and higher-level flash management.

Boot Stage	Location	Responsibilities
PBL — Primary Bootloader	ROM / Protected Flash	Power-on init, clock/watchdog setup, secure boot verification, SBL integrity check, jump to SBL
SBL — Secondary Bootloader	Reprogrammable Flash (SBL region)	Full UDS stack, flash driver execution, block erase/program, CRC/CMAC verification, jump to application
Application SW	Application Flash Region	Normal ECU function; cannot perform self-flash without SBL
Boot Flags / NvM	Non-volatile Memory (NvM)	BootReason, programmingRequest flag, last programming date, HW/SW compatibility matrix

8.2 Boot Mode Detection and Jump Logic

The bootloader determines the boot path at startup by inspecting a set of boot-mode flags stored in non-volatile RAM (NVRAM) or dedicated NvM blocks. The decision to remain in bootloader mode or jump to the application is governed by the following priority-ordered checks:

```
/* Boot Mode Decision Logic -- executed by PBL at power-on */
Boot_ModeDecision():
  // Priority 1: Dedicated programming-request flag (set by app before reset)
  IF (NvM_Read(BOOT_FLAG_PROGRAMMING_REQUEST) == TRUE):
    NvM_Write(BOOT_FLAG_PROGRAMMING_REQUEST, FALSE)
    GOTO BootloaderMode  // Enter SBL, await UDS session

  // Priority 2: Hardware pin / jumper (EOL bench programming)
  IF (GPIO_Read(BOOT_MODE_PIN) == ACTIVE_LOW):
    GOTO BootloaderMode

  // Priority 3: Application integrity check
  IF (NOT Crypto_VerifyApplicationCRC()):
    DTC_Set(DEM_EVENT_APP_INTEGRITY_FAIL)
    GOTO BootloaderMode  // App corrupted; stay in bootloader

  // Priority 4: Secure boot signature verification
  IF (NOT SecureBoot_VerifyAppSignature()):
    GOTO BootloaderMode  // Signature invalid; reject jump
```

```
    // All checks passed -- jump to application
    App_JumpToApplication(APP_START_ADDR)
```

8.3 UDS Bootloader Service Sequence

When the ECU is in bootloader mode, the SBL exposes a restricted UDS service set. Only services required for firmware programming are active; services such as RDBI (0x22) for OEM DIDs and FiM are unavailable until the application runs. The complete programming sequence per ISO 14229-1 and AUTOSAR SWS_DiagnosticCommunicationManager:

Step	UDS Service	Request / Response Example	Purpose
1	0x10 DiagnosticSessionControl	10 02 / 50 02 ...	Enter Programming Session; activates SBL full service set
2	0x27 SecurityAccess (0x01/0x02)	27 01 / 67 01 [SEED] then 27 02 [KEY]	Unlock programming security level (HSM seed/key)
3	0x28 CommunicationControl	28 03 01 / 68 03	Disable normal Rx/Tx to prevent bus collisions during erase
4	0x85 ControlDTCSetting	85 02 FF FF FF / C5 02	Disable DTC storage; suppress false faults during programming
5	0x31 RoutineControl 0xFF00	31 01 FF 00 [addr][len] / 71 01 FF 00 01	Erase target flash sector(s); async -- poll 0x31 0x03 for completion
6	0x34 RequestDownload	34 00 44 [memAddr 4B][size 4B] / 74 40 [maxBlock]	Specify download address, size, compression; ECU returns maxBlockLen
7	0x36 TransferData (N blocks)	36 01 [data...] / 76 01 (repeated)	Stream firmware blocks; blockSeqCtr 0x01..0xFF then wraps to 0x00
8	0x37 RequestTransferExit	37 / 77	Finalise transfer; ECU verifies CRC/CMAC of received image
9	0x31 0x01 0xFF01	31 01 FF 01 / 71 01 FF 01 00	CheckProgrammingDependencies: HW/SW compatibility validation
10	0x28 CommunicationControl	28 00 01 / 68 00	Re-enable normal bus communication
11	0x85 ControlDTCSetting	85 01 FF FF FF / C5 01	Re-enable DTC storage
12	0x11 ECUReset	11 01 / 51 01	Hard reset; PBL performs secure boot verify before jumping to new app

8.4 Flash Memory Management in Bootloader

Flash management in the SBL involves sector-level erase followed by word/double-word programming. The flash driver is typically downloaded into RAM before execution (to avoid XiP conflicts when erasing the region containing the flash driver itself). Key timing and NRC considerations:

Flash Operation	Typical Duration	NRC Handling
Sector Erase (64 KB sector)	200 ms - 2 s per sector	ECU sends 0x78 (ResponsePending) every P2 window; tester restarts P2* on each 0x78
Page Program (256 bytes)	1 - 5 ms	Inline; completed within P2 window; no 0x78 required
CRC/CMAC Verify (full image)	50 - 500 ms	Async routine (0x31 0xFF00); tester polls 0x31 0x03 until routineStatus = complete

Flash Driver Download to RAM	< 1 s	Standard 0x34/0x36/0x37 to RAM address; executed before main image download
Signature Verify (ECDSA/RSA)	10 - 100 ms	Performed by HSM after 0x37; result embedded in 0x77 or 0xFF01 routine status

8.5 Secure Boot and Cryptographic Validation

AUTOSAR SecureBoot and ISO 21434 mandate that firmware images are cryptographically signed and verified before execution. The verification chain ensures that only OEM-authorised software runs on safety-critical ECUs.

Mechanism	Standard / Module	Description
AUTOSAR SecureBoot	AUTOSAR SWS_SecureBoot	Verifies application binary against stored public key/certificate before jump; abort if fail
Uptane Metadata Verification	Uptane TUF framework	OTA: Director + Image repository metadata verified before any ECU install
CMAC (AES-128)	ISO 21434 / SHE/HSM	Block-level MAC; each flash block verified during TransferData for in-transfer integrity
ECDSA-256 Signature	NIST P-256 / HSM	Full image signature stored in NvM; verified by PBL at every power-on
Hash (SHA-256)	FIPS 180-4	Stored in manifest; used for quick integrity check prior to ECDSA verify
Rollback Protection	NvM monotonic counter	Prevents downgrade attack; programming fails if SW version <= installed version

```
/* Secure Boot Verification Flow (PBL) -- pseudocode */
SecureBoot_VerifyAppSignature():
  // 1. Read manifest from protected NvM region
  manifest = NvM_Read(NVM_BLOCK_APP_MANIFEST)

  // 2. Compute SHA-256 hash of application flash region
  computed_hash = HSM_SHA256(APP_FLASH_START, APP_FLASH_SIZE)

  // 3. Compare hash against manifest
  IF (computed_hash != manifest.expected_hash):
    return SECURE_BOOT_FAIL_HASH

  // 4. Verify ECDSA-256 signature using OEM public key (in ROM)
  result = HSM_ECDSA_Verify(
    signature  = manifest.ecdsa_signature,
    message    = computed_hash,
    public_key = OEM_PUBLIC_KEY_ROM
  )
  IF (result != CRYPTO_SUCCESS):
    return SECURE_BOOT_FAIL_SIGNATURE

  // 5. Check rollback protection
  IF (manifest.sw_version <= NvM_Read(NVM_BLOCK_INSTALLED_VERSION)):
    return SECURE_BOOT_FAIL_ROLLBACK

  return SECURE_BOOT_PASS
```

8.6 Bootloader NRC Handling and Programming Preconditions

Programming preconditions prevent inadvertent reprogramming in unsafe vehicle states. Violating preconditions causes the ECU to respond with NRC 0x22 (conditionsNotCorrect). Preconditions are typically

checked by RoutineControl 0xFF03 (checkProgrammingPreconditions) before the main programming sequence:

Precondition	Check Mechanism	NRC if Violated
Vehicle speed == 0 km/h	CAN signal from ABS/ESC ECU via gateway	0x22 conditionsNotCorrect
Engine not running (powertrain ECUs)	RPM signal == 0 or keyOff state	0x22 conditionsNotCorrect
Battery voltage in range (11-15 V)	ADC measurement via power supervisor	0x22 conditionsNotCorrect
No active safety-critical DTCs	DEM query: no ASIL-D confirmed DTCs	0x22 conditionsNotCorrect
Programming session correctly entered	Bootloader internal session flag	0x7F serviceNotSupportedInActiveSession
Security Access granted (level 1)	DCM security state machine	0x33 securityAccessDenied
Flash erase completed before program	Bootloader sequence flag	0x24 requestSequenceError
Block sequence counter correct	SBL internal counter	0x3B wrongBlockSequenceCounter
Transfer CRC matches expected	HSM/CRC peripheral result	0x3A generalProgrammingFailure

CHAPTER 9

SAE J1939 Heavy-Duty Vehicle Diagnostics

SAE J1939 is the dominant diagnostic and application-data network standard for heavy-duty trucks, buses, agricultural machinery, and off-highway equipment. It defines the network, transport, application, and diagnostic layers over CAN (250 kbps / 500 kbps), and forms the basis for HD-OBD regulatory requirements in the USA (EPA/CARB) and Europe (Euro VI).

9.1 J1939 Architecture Overview

J1939 uses a 29-bit extended CAN identifier that encodes the Priority, Reserved bit, Data Page, Protocol Data Unit (PDU) Format, PDU Specific (destination or group extension), and Source Address. This ID structure enables broadcast (PDU1 with destination 0xFF), peer-to-peer (PDU1, specific destination), and proprietary (PDU2) messaging within a single network.

```
/* J1939 CAN 29-bit Extended Identifier Structure */
+---+--+--+--------+--------+--------+
| P | R | DP | PF     | PS/DA  | SA     |
| 3b| 1 | 1  | 8 bits | 8 bits | 8 bits |
+---+--+--+--------+--------+--------+
P  = Priority (0=highest, 7=lowest; diagnostic = 6)
R  = Reserved (always 0)
DP = Data Page (0=J1939 defined, 1=proprietary/NMEA)
PF = PDU Format: < 0xF0 => PDU1 (peer-to-peer)
                 >= 0xF0 => PDU2 (broadcast group extension)
PS = PDU Specific: PDU1 => Destination Address (DA)
                   PDU2 => Group Extension (GE)
SA = Source Address (unique per ECU: 0x00-0xFD)

/* Parameter Group Number (PGN) = R + DP + PF + GE (PDU2) */
/* or R + DP + PF + 0x00 (PDU1, destination excluded) */

/* Example: DM1 Active DTCs, broadcast PGN 0xFECA */
CAN ID (29-bit): 0x18FECA00
  Priority = 6 (110b), DP=0, PF=0xFE, PS=0xCA, SA=0x00
  PGN = 0x00FECA = 65226 decimal
```

9.2 J1939 Address Structure and ECU Addressing

Address Range	Category	Description
0x00	Engine Controller #1	Primary engine ECM (tractor unit)
0x00-0x7F	Industry-standard ECUs	SAE J1939-71 assigned addresses for standard controllers
0x80-0xF7	Self-configurable / dynamic	Address Claiming procedure (NAME arbitration) per J1939-81
0xF8-0xFD	Manufacturer specific	OEM or system supplier defined devices
0xFE	Null Address	Used temporarily during address claiming before claim resolution
0xFF	Global / Broadcast	Destination for broadcast messages; all ECUs receive

9.3 J1939 DTC Format and FMI

J1939 DTCs use a different encoding from OBD-II. Each J1939 fault consists of a Suspect Parameter Number (SPN) identifying the signal/component, a Failure Mode Identifier (FMI) describing the fault type, and an Occurrence Counter. The DTC is encoded in 4 bytes within Diagnostic Messages (DMs):

```
/* J1939 DTC Encoding in DM1/DM2 message payload */
Bytes 0-3 per DTC (SPN + FMI + OC + CM):
  Byte 0 (bits 7-0): SPN[7:0]   -- SPN lower 8 bits
  Byte 1 (bits 7-0): SPN[15:8]  -- SPN middle 8 bits
  Byte 2 (bits 7-5): SPN[18:16] -- SPN upper 3 bits
         (bits 4-0): FMI[4:0]   -- Failure Mode Identifier
  Byte 3 (bit 7):     CM        -- Conversion Method flag
         (bits 6-0): OC[6:0]    -- Occurrence Count (0-127)

/* Example: SPN 91 (Accelerator Pedal Position 1), FMI 3 (Short to VCC) */
  SPN = 91 = 0x00005B
  FMI = 3
  OC  = 12
  Bytes: 5B 00 03 0C  (little-endian SPN + FMI + OC)
```

FMI	Failure Mode Identifier Name	Description
0	Data Valid But Above Normal Range (Most Severe)	Signal above calibrated maximum range
1	Data Valid But Below Normal Range (Most Severe)	Signal below calibrated minimum range
2	Data Erratic / Intermittent / Incorrect	Signal present but intermittent or shows incorrect values
3	Voltage Above Normal / Short to High	Signal voltage above expected range; short to supply
4	Voltage Below Normal / Short to Low	Signal voltage below expected range; short to ground
5	Current Below Normal / Open Circuit	Below-normal current indicating open circuit condition
6	Current Above Normal / Grounded Circuit	Excessive current; short-to-ground or shorted actuator
7	Mechanical System Not Responding Properly	Mechanical jam, stall, or actuator not reaching setpoint
8	Abnormal Frequency, Pulse Width, or Period	Signal frequency out of expected range
9	Abnormal Update Rate	Message not received within expected time window
10	Abnormal Rate of Change	Signal changing too rapidly; gradient fault
11	Root Cause Not Known	Fault detected but specific failure mode cannot be isolated
12	Bad Intelligent Device or Component	ECU internal fault; requires module replacement
13	Out of Calibration	Component out of calibration; requires re-calibration
14	Special Instructions	Follow service manual special instructions
15	Data Valid But Above Normal Range (Least Severe)	Signal above normal range, least severe classification
19	Received Network Data in Error	Received J1939 message data contains CRC or format error
31	Not Available / Not Installed	Parameter not available or component not installed

9.4 J1939-73 Diagnostic Messages (DMs)

SAE J1939-73 defines a comprehensive set of Diagnostic Messages (DM1 through DM35+) covering active faults, stored faults, freeze frames, readiness, and test results. Each DM uses a specific PGN and is either broadcast periodically or sent on request:

DM #	PGN	Transmission	Content / Purpose
DM 1	0xFECA (65226)	1 Hz broadcast	Active diagnostic trouble codes and lamp status (MIL/AWL/RSL/PL)
DM 2	0xFECB (65227)	On-Request	Previously active (stored) DTCs -- equivalent to OBD Mode $03
DM 3	0xFECC (65228)	On-Request	Clear previously active DTCs -- equivalent to OBD Mode $04
DM 4	0xFECD (65229)	On-Request	Freeze frame parameters captured when DTC became active
DM 5	0xFECE (65230)	On-Request	Readiness status of diagnostic monitors -- equivalent to OBD PID 0x01
DM 6	0xFECF (65231)	On-Request	Emission-related pending DTCs -- equivalent to OBD Mode $07
DM 11	0xFED3 (65235)	On-Request	Clear active and previously active DTCs -- full DTC clear
DM 12	0xFED4 (65236)	On-Request	Emission-related active DTCs
DM 20	0xFEDC (65244)	On-Request	Ratio of monitor runs to ignition cycles (OBD-equivalent IUPR)
DM 21	0xFEDD (65245)	On-Request	Diagnostic readiness: distance/time with MIL on since last clear
DM 22	0xFEDE (65246)	On-Request	Individual clear/reset of active DTCs for specific SPN/FMI
DM 25	0xFEE1 (65249)	On-Request	Expanded freeze frame -- more parameters than DM4
DM 28	0xFEE4 (65252)	On-Request	Permanently active DTCs -- equivalent to OBD Mode $0A
DM 30	0xFEE6 (65254)	On-Request	Scaled test results for specific SPN/FMI -- equivalent to OBD Mode $06
DM 31	0xFEE7 (65255)	On-Request	DTC to lamp mapping -- identifies which lamp is controlled by each DTC

9.5 DM1 Message — Active DTCs and Lamp Status

DM1 is the most critical J1939 diagnostic message. It is broadcast at 1 Hz on PGN 0xFECA and reports all currently active DTCs plus the lamp command byte. Controllers monitoring the J1939 bus can react to DM1 in real time without polling.

```
/* DM1 Message Structure (PGN 0xFECA, up to 1785 bytes via BAM) */
Byte 0: Lamp Status (4 nibbles):
  Bits 7-6: MIL status (00=off, 01=on, 10=flash, 11=n/a)
  Bits 5-4: RSL status (00=off, 01=on, 10=flash, 11=n/a)
  Bits 3-2: AWL status (00=off, 01=on, 10=flash, 11=n/a)
  Bits 1-0: PL  status (00=off, 01=on, 10=flash, 11=n/a)
Byte 1: Flash Status (lamp blink pattern; mirrors Byte 0 for flash rates)
Bytes 2+ (4 bytes per DTC, up to 250 DTCs):
  [SPN_lo][SPN_mid][SPN_hi_FMI][CM_OC]   -- see Section 9.3

/* Example DM1: MIL on, one active DTC */
CAN ID: 0x18FECA0B  (PGN 0xFECA, SA=0x0B = Transmission)
Data: 40 FF 5B 00 43 8C  (6 bytes)
  Byte 0 = 0x40: MIL=ON(01), RSL=OFF(00), AWL=OFF(00), PL=OFF(00)
  Byte 1 = 0xFF: no flash (all off)
```

```
    Bytes 2-5: DTC: SPN=91(0x5B), FMI=3, CM=1, OC=12
```

9.6 J1939 Transport Protocol — BAM and CMDT

J1939 data frames are limited to 8 bytes. Messages exceeding 8 bytes use the J1939-21 Transport Protocol (TP), which provides two modes: Broadcast Announce Message (BAM) for global destinations and Connection Mode Data Transfer (CMDT) for peer-to-peer communication. DM1 with many DTCs and DM4 freeze frames frequently use BAM.

TP Phase	PGN / Identifier	Description
TP.CM_BAM (Connection Management -- Broadcast)	PGN 0xEC00 (60416)	Announces upcoming multi-packet broadcast: total bytes, number of packets, PGN of data
TP.DT (Data Transfer)	PGN 0xEB00 (60160)	Carries sequential 7-byte chunks; byte 1 = sequence counter (1..255)
TP.CM_RTS (Request to Send)	PGN 0xEC00 (60416)	CMDT: initiator requests peer-to-peer session; includes total size
TP.CM_CTS (Clear to Send)	PGN 0xEC00 (60416)	CMDT: responder grants transfer for N packets from packet M
TP.CM_EndofMsgAck	PGN 0xEC00 (60416)	CMDT: acknowledges successful receipt of complete message
TP.Conn_Abort	PGN 0xEC00 (60416)	Aborts the transport session; includes abort reason code

9.7 HD-OBD Requirements and J1939 Regulatory Integration

Heavy-Duty OBD (HD-OBD) in the USA (40 CFR Part 86/1065) and EU (Euro VI Regulation 582/2011) mandates SAE J1939-based OBD for vehicles over 3860 kg GVWR. Key differences from light-duty OBD-II include additional diagnostic monitors, J1939 transport (instead of ISO 15765-2), and stricter readiness requirements for NOx and PM systems.

Requirement	Light-Duty OBD-II	HD-OBD (J1939-73)
Network protocol	ISO 15765-4 CAN TP (500 kbps)	SAE J1939 (250/500 kbps)
DTC format	SAE J2012 (P/C/B/U codes)	SPN + FMI encoding (J1939-73)
Active DTC message	OBD Mode $03 (request-based)	DM1 (1 Hz broadcast automatic)
Monitor status message	Mode $01 PID 0x01	DM5 (on-request)
Freeze frame	Mode $02 (on-request)	DM4 / DM25 (on-request, more params)
Readiness time/distance	PIDs 0x4D/0x4E	DM21 (distance with MIL, time with MIL)
IUPR monitoring ratio	Not mandated (CARB optional)	DM20 mandatory (Euro VI / US HD-OBD)
Clear DTCs	Mode $04 (full clear)	DM3 (stored), DM11 (all), DM22 (per DTC)
NOx monitor	NCAT non-continuous	NOx catalyst + SCR mandatory continuous
PM filter monitor	PM non-continuous	DPF mandatory continuous with DM30 test results

9.8 J1939 vs UDS Coexistence in Modern Heavy-Duty ECUs

Modern heavy-duty powertrain ECUs often implement both J1939 (for fleet diagnostics and telematics) and UDS/ISO 14229 (for dealer-level deep diagnostics, flashing, and calibration). The gateway ECU routes messages between the external OBD port and the internal J1939 backbone. The DCM module handles UDS while a separate J1939 diagnostic application manages DM1-DM35 broadcast and response services.

```
/* J1939 + UDS Coexistence Architecture */
External OBD Port (J1962 / DoIP Ethernet)
  |
  | UDS (ISO 14229) on CAN 0x7E0/0x7E8 or DoIP
  v
  Gateway / Central ECU
  |                    |
  | J1939 (250 kbps)   | UDS routing to sub-ECUs (ISO 14229-4)
  v                    v
  Engine ECM           Aftertreatment ECU
  (J1939 SA=0x00)      (J1939 SA=0x04)
  |                    |
  DM1 broadcast 1 Hz   DM1 broadcast 1 Hz
  DM5 on-request       DM30 NOx test results
  UDS 0x22/0x2E/0x31   UDS flash programming
```

CHAPTER 10

Diagnostic Event Manager (DEM)

The Diagnostic Event Manager (DEM) is the AUTOSAR BSW module responsible for monitoring, storing, and managing all diagnostic faults (events) in an ECU. It implements the fault lifecycle from initial detection through confirmation, storage in non-volatile memory, aging, healing, and eventual removal. DEM is defined in AUTOSAR_SWS_DiagnosticEventManager.

10.1 DEM Architecture Overview

```
+-----------------------------------------------+
|            APPLICATION SWCs (ASW)             |
|  Monitors -> Dem_SetEventStatus()             |
+------+-------+------+------------------------+
       |       |      |  Rte_Call_...()
       v       v      v
+-----------------------------------------------+
|             DEM CORE                          |
|  +------------------+  +------------------+ |
|  | Event Processing |  | DTC Management   | |
|  | - Debounce       |  | - Confirmation   | |
|  | - Status Update  |  | - Memory Mgmt    | |
|  +------------------+  | - Aging/Healing  | |
|  +------------------+  +------------------+ |
|  | Indicator Ctrl   |  | OBD Manager      | |
|  | - MIL, AWL, PL   |  | - Mode $01/$03   | |
|  | - WIR bit        |  | - Readiness      | |
|  +------------------+  +------------------+ |
+------+-----+-----------+--------------------+
       |     |           |
       v     v           v
+------+ +------+  +----------+
| NvM  | | FIM  |  | DCM / SW |
| Block| |(Inhibit) |(requests)|
+------+ +------+  +----------+
```

10.2 Event Status Reporting

Application SWCs report diagnostic events using Dem_SetEventStatus(). The event ID maps to a DTC in the DEM configuration:

```
// DEM Event Status Values
#define DEM_EVENT_STATUS_PASSED    0x00  // Monitor completed: no fault detected
#define DEM_EVENT_STATUS_FAILED    0x01  // Monitor completed: fault detected
#define DEM_EVENT_STATUS_PREPASSED 0x02  // Pre-qualifier: towards PASSED (counter-
based)
#define DEM_EVENT_STATUS_PREFAILED 0x03  // Pre-qualifier: towards FAILED (counter-
based)

// Application SWC Example: Engine Temperature Monitor
FUNCTION EngineTemp_Monitor_MainFunction():
  temp = ReadSensor(SENSOR_ECT)

  IF (temp > 120 && temp < 150):
    // Valid reading within warning range
    Dem_SetEventStatus(DemConf_DemEventParameter_ECT_WarnRange,
                       DEM_EVENT_STATUS_FAILED)
```

```
  ELSE IF (temp >= 150):
    // Sensor range exceeded - plausibility error
    Dem_SetEventStatus(DemConf_DemEventParameter_ECT_SensorFail,
                       DEM_EVENT_STATUS_FAILED)

  ELSE IF (temp < 0):
    // Open circuit / short to ground
    Dem_SetEventStatus(DemConf_DemEventParameter_ECT_OpenCircuit,
                       DEM_EVENT_STATUS_FAILED)

  ELSE:
    // All OK - report PASSED for all monitors
    Dem_SetEventStatus(DemConf_DemEventParameter_ECT_WarnRange,
                       DEM_EVENT_STATUS_PASSED)
    Dem_SetEventStatus(DemConf_DemEventParameter_ECT_SensorFail,
                       DEM_EVENT_STATUS_PASSED)
    Dem_SetEventStatus(DemConf_DemEventParameter_ECT_OpenCircuit,
                       DEM_EVENT_STATUS_PASSED)
```

10.3 Debounce Algorithms

DEM debounce algorithms prevent transient sensor errors from immediately setting DTCs. AUTOSAR supports two standard algorithms plus custom extensions:

10.3.1 Counter-Based Debounce

```
// Counter-based debounce (DEM_DEBOUNCE_COUNTER_BASED)
// FDC: Fault Detection Counter, range: -128 (passed) to +127 (failed)
// Parameters: DemDebounceCounterFailedThreshold (e.g., +127)
//             DemDebounceCounterPassedThreshold (e.g., -128)
//             DemDebounceCounterIncrementStepSize (e.g., +10)
//             DemDebounceCounterDecrementStepSize (e.g., -5)

FUNCTION DEM_CounterDebounce(event, status):
  IF (status == DEM_EVENT_STATUS_PREFAILED):
    event.FDC = MIN(event.FDC + IncrementStep, FDCMax)

  ELIF (status == DEM_EVENT_STATUS_PREPASSED):
    event.FDC = MAX(event.FDC - DecrementStep, FDCMin)

  ELIF (status == DEM_EVENT_STATUS_FAILED):
    event.FDC = FDCMax  // Jump to max (immediately failed)

  ELIF (status == DEM_EVENT_STATUS_PASSED):
    event.FDC = FDCMin  // Jump to min (immediately passed)

  IF (event.FDC >= DemDebounceCounterFailedThreshold):
    DEM_SetInternalStatus(event, TEST_FAILED)
    event.qualifiedStatus = DEM_EVENT_STATUS_FAILED

  ELIF (event.FDC <= DemDebounceCounterPassedThreshold):
    DEM_SetInternalStatus(event, TEST_PASSED)
    event.qualifiedStatus = DEM_EVENT_STATUS_PASSED
```

10.3.2 Time-Based Debounce

```
// Time-based debounce (DEM_DEBOUNCE_TIME_BASED)
// Parameters: DemDebounceTimeFailedThreshold (e.g., 500ms)
//             DemDebounceTimePassedThreshold (e.g., 200ms)
```

```
FUNCTION DEM_TimeDebounce(event, status):
  IF (status == DEM_EVENT_STATUS_FAILED OR PREFAILED):
    event.failedTimer += DemMainFunctionPeriod
    event.passedTimer = 0
    IF (event.failedTimer >= FailedThreshold):
      DEM_SetInternalStatus(event, TEST_FAILED)

  ELIF (status == DEM_EVENT_STATUS_PASSED OR PREPASSED):
    event.passedTimer += DemMainFunctionPeriod
    event.failedTimer = 0
    IF (event.passedTimer >= PassedThreshold):
      DEM_SetInternalStatus(event, TEST_PASSED)
```

10.4 DTC Confirmation and Storage

```
FUNCTION DEM_ProcessQualifiedStatus(event):

  // --- FAILED path ---
  IF (qualifiedStatus == TEST_FAILED):

    // Update status byte bits 0, 1
    SET event.statusByte |= BIT0_TF         // testFailed
    SET event.statusByte |= BIT1_TFTOC       // testFailedThisOpCycle
    SET event.statusByte |= BIT5_TFSLC       // testFailedSinceLastClear
    CLEAR event.statusByte &= ~BIT6_TNCTOC   // test completed this cycle
    CLEAR event.statusByte &= ~BIT4_TNCTSLC  // test completed since clear

    // Pending DTC (bit 2)
    SET event.statusByte |= BIT2_PDTC

    // Confirmation check
    IF NOT (event.statusByte & BIT3_CDTC):  // not yet confirmed
      event.pendingCycleCounter++
      IF (event.pendingCycleCounter >= DemEventConfirmationCycles):
        SET event.statusByte |= BIT3_CDTC    // confirmedDTC
        DEM_StoreEventInPrimaryMemory(event) // store to NvM
        DEM_UpdateFreezeFrame(event)
        DEM_TriggerWarningIndicator(event)
        SET event.statusByte |= BIT7_WIR     // warningIndicatorRequested

  // --- PASSED path ---
  ELIF (qualifiedStatus == TEST_PASSED):

    CLEAR event.statusByte &= ~BIT0_TF       // testFailed cleared
    CLEAR event.statusByte &= ~BIT6_TNCTOC   // test completed
    CLEAR event.statusByte &= ~BIT4_TNCTSLC

    // Start healing if DTC was confirmed
    IF (event.statusByte & BIT3_CDTC):
      event.healingCycleCounter++
      IF (event.healingCycleCounter >= DemAgingCycleCounterThreshold):
        CLEAR event.statusByte &= ~BIT3_CDTC  // confirmed cleared
        CLEAR event.statusByte &= ~BIT7_WIR   // indicator cleared
        DEM_RemoveDTCFromPrimaryMemory(event)
```

10.5 DTC Lifecycle - Aging and Healing

DTC aging and healing follow a state machine driven by operation cycles (typically ignition cycles):

```
// DTC Lifecycle State Machine
```

```
+-----------------------------------------+
| INITIAL STATE (no DTC stored)           |
+---+-------------------------------------+
    | testFailed (FDC reaches max)
    v
+-----------------------------------------+
| PENDING_DTC                             |
| status byte: bit2=1, bit3=0             |
| Visible in Mode $07 / subFunc 0x07      |
+---+-------+-----------------------------+
    |       |
    | Fails again in next op cycle
    | pendingCycleCounter >= threshold
    v
+-----------------------------------------+
| CONFIRMED_DTC                           |
| status byte: bit2=1, bit3=1, bit7=1(MIL) |
| Stored in primary memory                |
| Snapshot captured, occurrence++ increment |
+---+-------------------------------------+
    |
    | Passes for DemAgingCycleCounterThreshold consecutive cycles
    | (each ignition-on/off where test completes PASSED)
    v
+-----------------------------------------+
| AGED / HEALED                           |
| status byte: bit3=0, bit7=0             |
| DTC removed from primary memory         |
| MIL extinguished (if no other DTCs)     |
+-----------------------------------------+

// Key DEM configuration parameters:
// DemOperationCycleRef      -> references operation cycle (e.g., IGNITION)
// DemAgingCycleRef          -> cycle used for aging (often same as op cycle)
// DemAgingCycleCounterThreshold -> number of passed cycles to age out DTC (e.g., 40)
// DemEventConfirmationCycles -> cycles before pending becomes confirmed (default 1)
```

10.6 DEM Event Memory Management

Memory Type	Max DTCs	Clearable?	Use Case
Primary Memory	OEM-defined (e.g., 25-100)	Yes (0x14)	Standard DTC storage for all confirmed events
Mirror Memory	Subset of primary	Yes (RID 0xFF02)	Read-only snapshot of last confirmed DTCs for legal/audit
Permanent Memory	OBD emission DTCs only	No (only via driving cycles)	WWH-OBD: DTCs survive Mode 04 / 0x14 until healed
User-Defined Memory	OEM-configurable	Via custom RID	AUTOSAR R21-11+: separate memory partition per use case

10.7 DEM Key API Reference

API Function	Direction	Description
Dem_SetEventStatus(EventId, Status)	SWC->DEM	Report monitor result (PASSED/FAILED/PRE...)
Dem_GetEventStatus(EventId, *StatusByte)	SWC->DEM	Read current event status byte
Dem_GetEventFailed(EventId, *EventFailed)	SWC->DEM	Quick boolean: is event currently failed?

API Function	Direction	Description
Dem_GetEventTested(EventId, *EventTested)	SWC->DEM	Has event completed at least one test run?
Dem_GetFaultDetectionCounter(EventId, *FDC)	SWC->DEM	Read FDC (-128 to +127)
Dem_GetDTCOfEvent(EventId, Format, *DTC)	SWC->DEM	Get 3-byte DTC associated with event
Dem_GetStatusOfDTC(DTC, Origin, *Status)	DCM->DEM	Read DTC status byte by DTC number
Dem_ClearDTC(DTC, Format, Origin)	DCM->DEM	Clear DTC and associated data
Dem_SetDTCFilter(...)	DCM->DEM	Set filter for GetNextFilteredDTC()
Dem_GetNextFilteredDTC(*DTC, *Status)	DCM->DEM	Get next DTC matching filter
Dem_GetIndicatorStatus(IndicatorId, *Status)	BSW->DEM	Get indicator (MIL/AWL/PL) status for output
Dem_GetDTCExtendedDataRecordByDTCNumber()	DCM->DEM	Read extended data for given DTC
Dem_GetDTCSnapshotRecordByDTCNumber()	DCM->DEM	Read freeze frame data for given DTC
Dem_SetEventDisabled(EventId)	SWC->DEM	Temporarily disable event testing
Dem_EnableDTCRecordUpdate(DTCGroup, Origin)	DCM->DEM	Re-enable DTC update after freeze for read

CHAPTER 11

DCM — Advanced Configuration & Callout APIs

The Diagnostic Communication Manager (DCM) is an AUTOSAR Basic Software (BSW) module that implements the complete UDS and OBD service stack. It mediates between the communication stack (CAN TP, DoIP) and application SWCs via the RTE.

11.1 DCM Internal Architecture (Three Sub-Layers)

DCM is internally structured into three functional sub-layers:

```
+-----------------------------------------------------------+
|              DCM - Diagnostic Communication Manager       |
+------------------+------------------+--------------------+
|  DSL             |  DSD             |  DSP               |
| Diagnostic       | Diagnostic       | Diagnostic         |
| Session Layer    | Service Dispatch | Service Processor  |
+------------------+------------------+--------------------+
| - Session mgmt   | - SID routing    | - Service handlers |
| - Security state | - SubFunc route  | - DID read/write   |
| - Timing (P2/S3) | - Session/Sec    | - RID execution    |
| - Addressing     |   validation     | - DTC access       |
+------------------+------------------+--------------------+
          |                 |                   |
          v                 v                   v
+-----------------------------------------------------------+
|           PduR (PDU Router) -> CanTp / DoIP               |
+-----------------------------------------------------------+
```

11.2 DCM AUTOSAR Configuration Containers

ARXML Container	Key Parameters	Purpose
DcmDsp	DcmDspDid, DcmDspPid, DcmDspRoutine, DcmDspSecurity	Core service processor config
DcmDspDid	DcmDspDidIdentifier, DcmDspDidDataRef	Each DID definition and callbacks
DcmDspData	ReadDataFnc, WriteDataFnc, DataSize, UsePort	DID data accessor functions
DcmDspRoutine	DcmDspRoutineIdentifier, StartFnc, StopFnc, ResultsFnc	RID handler functions
DcmDspSecurity	SecurityLevel, GetSeedFnc, CompareKeyFnc, AttemptCounter	Security access config
DcmDspSession	SessionLevel, P2ServerMax, P2StarServerMax	Session timing and access
DcmDsl	DcmDslProtocol, P2ServerMax, P2StarServerMax, S3Server	Transport and session timing
DcmDslBuffer	DcmDslBufferSize	Rx/Tx buffer sizes (max 4095 bytes)

11.3 DCM Callout API — Application Hooks

```
/* ReadData callout for DID 0xF190 (VIN) */
Std_ReturnType App_ReadDID_VIN(
    uint8* Data,
    Dcm_NegativeResponseCodeType* ErrorCode) {
```

```
  if (NvM_ReadBlock(NVM_BLOCK_VIN, Data) != E_OK) {
    *ErrorCode = DCM_E_GENERALREJECT;
    return E_NOT_OK;
  }
  return E_OK;
}

/* SecurityAccess: Provide Seed (HSM-backed RNG) */
Std_ReturnType App_GetSeed(
    uint8* Seed, uint16* SeedLen,
    Dcm_NegativeResponseCodeType* ErrorCode) {
  uint32_t rnd = Rng_GetHardwareRandom();
  memcpy(Seed, &rnd, 4);
  *SeedLen = 4;
  SecurityCtx.pendingSeed = rnd;
  return E_OK;
}

/* SecurityAccess: Compare Key */
Std_ReturnType App_CompareKey(
    const uint8* Key,
    Dcm_NegativeResponseCodeType* ErrorCode) {
  uint32_t expected = OEM_ComputeKey(SecurityCtx.pendingSeed);
  uint32_t rcvd; memcpy(&rcvd, Key, 4);
  if (rcvd != expected) { *ErrorCode = DCM_E_INVALIDKEY; return E_NOT_OK; }
  return E_OK;
}
```

11.4 S3 Timer and Session Maintenance

```
/* S3 timer logic — DCM MainFunction (5 ms task) */
void Dcm_MainFunction(void) {
  if (currentSession != DEFAULT_SESSION) {
    s3Timer -= TASK_PERIOD_MS;
    if (s3Timer <= 0) {
      Dcm_TransitionToDefaultSession();
      s3Timer = DCM_S3_SERVER_MAX;  /* 5000 ms */
    }
  }
}

/* Any valid request resets S3 timer */
void Dcm_OnValidRequestReceived(void) {
  s3Timer = DCM_S3_SERVER_MAX;
}
```

TesterPresent Session Keep-Alive

Send [0x3E][0x80] (suppressPosRsp) every 1500 ms to reset S3 without generating bus traffic. Most tester tools send this automatically in the background when in a non-default session.

CHAPTER 12

DEM — Debounce Algorithms, Storage & Healing

The Diagnostic Event Manager (DEM) is the central AUTOSAR BSW module responsible for processing and storing all diagnostic events detected by application SWCs. DEM decouples fault detection from DTC storage, status management, and indicator control.

12.1 DEM Event Processing Pipeline

```
        SWC Monitor              DEM Core              NvM
             |                      |                  |
  Dem_SetEventStatus(FAILED)        |                  |
            +---------------------->|                  |
                                Debounce Algorithm     |
                                (Counter or Timer)     |
                                      |                |
                               [Qualified FAILED?]     |
                                      |                |
                                Update Status Byte     |
                                Capture Snapshot Data  |
                                Store DTC to NvM  ---------->|
                                      |                |
                               Notify FiM + HMI (MIL)  |
```

12.2 DEM Debounce Algorithms

12.2.1 Counter-Based Debounce

Parameter	Description	Typical Value
DemDebounceCounterFailedThreshold	FDC (+127) = qualified FAILED	+3 to +10
DemDebounceCounterPassedThreshold	FDC (-128) = qualified PASSED	-3 to -10
DemDebounceCounterIncrementStepSize	Increment per test-failed	+1
DemDebounceCounterDecrementStepSize	Decrement per test-passed	-1
DemDebounceCounterJumpDown	Rapid drop to min threshold	FALSE

12.2.2 Time-Based Debounce

Parameter	Description	Typical Value
DemDebounceTimeFailedThreshold	Continuous fail time before FAILED	500 ms – 2 s
DemDebounceTimePassedThreshold	Continuous pass time before PASSED	100 ms – 1 s
DemDebounceTimerReset	Reset timer on interruption	TRUE

```
/* Counter-based debounce pseudocode */
void Dem_UpdateDebounce(Dem_Debounce_t* db, Dem_EventStatus status) {
  if (status == DEM_EVENT_STATUS_FAILED) {
    db->counter = MIN(db->counter + 1, db->failThreshold);
    if (db->counter >= db->failThreshold)
      Dem_QualifyEvent(DEM_EVENT_STATUS_FAILED);
  } else if (status == DEM_EVENT_STATUS_PASSED) {
```

Parameter	Description	Typical Value

```
    db->counter = MAX(db->counter - 1, db->passThreshold);
    if (db->counter <= db->passThreshold)
      Dem_QualifyEvent(DEM_EVENT_STATUS_PASSED);
  }
}
```

12.3 DTC Aging and Healing

Phase	Trigger	DTC Status Changes
Fault Detection	testFailed first time	TF=1, TFTOC=1, PDTC=1
Confirmation	Failed 2nd consecutive trip	CDTC=1, WIR=1, MIL ON
Healing Start	First passing test result	TF=0
Healing Complete	3 consecutive passing trips	WIR=0, MIL OFF
Aging Complete	40 operation cycles since healed	CDTC=0, DTC removed
Manual Clear	UDS 0x14 / OBD Mode 04	All bits cleared, DTC deleted

12.4 DEM Key API Reference

API Function	Direction	Description
Dem_SetEventStatus(EventId, Status)	SWC->DEM	Report monitor result PASSED/FAILED/PREFAILED/PREPASSED
Dem_GetEventStatus(EventId, *Status)	SWC->DEM	Read current event status byte
Dem_GetFaultDetectionCounter(EventId, *FDC)	SWC->DEM	Read FDC value (-128..+127)
Dem_GetDTCOfEvent(EventId, Format, *DTC)	SWC->DEM	Get 3-byte DTC number for event
Dem_GetStatusOfDTC(DTC, Origin, *Status)	DCM->DEM	Read DTC status byte by DTC number
Dem_ClearDTC(DTC, Format, Origin)	DCM->DEM	Clear DTC and associated snapshot/extended data
Dem_SetDTCFilter(mask, ...)	DCM->DEM	Set filter for GetNextFilteredDTC()
Dem_GetNextFilteredDTC(*DTC, *Status)	DCM->DEM	Iterate DTCs matching filter (for 0x19)
Dem_GetIndicatorStatus(IndicatorId, *Status)	BSW->DEM	Get MIL/AWL indicator output status
Dem_GetDTCSnapshotRecordByDTCNumber()	DCM->DEM	Read freeze-frame data for DTC

CHAPTER 13

RTE — Runtime Environment in Diagnostics

The Runtime Environment (RTE) is the AUTOSAR-generated middleware that implements port connections between SWCs and BSW modules. In diagnostics, the RTE connects fault monitors to DEM, DID providers to DCM, and FiM permission checks to application SWCs.

13.1 RTE Port Types Used in Diagnostics

Port Type	Communication Model	Diagnostic Use
Sender-Receiver (S/R)	Asynchronous data flow	SWC sends fault status; DEM receives monitor result
Client-Server (C/S)	Synchronous function call	DCM calls SWC read DID; SWC executes and returns data
Mode-Switch	Mode declaration/reception	DEM notifies SWC of DTC lifecycle mode changes
Trigger-Receiver	Event-based notification	DEM triggers SWC on DTC state confirmation

13.2 SWC to DEM Fault Reporting via RTE

```
/* Application SWC fault monitor — AUTOSAR RTE API */
#include "Rte_FaultMonitor_SWC.h"

void FaultMonitor_20ms_Task(void) {
  float32 sensorVoltage;
  (void)Rte_Read_SensorVoltage_Port(&sensorVoltage);

  if (sensorVoltage < SENSOR_MIN_V || sensorVoltage > SENSOR_MAX_V) {
    (void)Rte_Call_DEM_EVENT_0x040500_SetEventStatus(
                        DEM_EVENT_STATUS_FAILED);
  } else {
    (void)Rte_Call_DEM_EVENT_0x040500_SetEventStatus(
                        DEM_EVENT_STATUS_PASSED);
  }
}
```

13.3 RTE DCM Callout — DID Read Implementation

```
/* DCM to SWC read callout via RTE — DID 0x1234 */
Std_ReturnType App_ReadDID_0x1234(
    P2VAR(uint8, AUTOMATIC, RTE_APPL_VAR) Data
) {
  BattState_t battState;
  (void)Rte_IRead_BattMgmt_Runnable_BattState(&battState);

  Data[0] = (uint8_t)(battState.stateOfCharge);  /* 0-100% */
  Data[1] = (uint8_t)(battState.voltage >> 8);   /* V high */
  Data[2] = (uint8_t)(battState.voltage & 0xFF); /* V low  */
  Data[3] = (uint8_t)(battState.tempCelsius + 40); /* offset */

  return RTE_E_OK;
}
```

13.4 FiM — Function Inhibition Manager

FiM uses DEM event status to automatically inhibit application features when faults are active. This prevents unsafe operation of subsystems with invalid inputs:

```
/* FiM permission query in engine control SWC */
void Engine_ControlRunnable(void) {
  boolean permission;

  Rte_Call_FiM_KNOCK_CONTROL_GetFunctionPermission(&permission);

  if (permission == TRUE) {
    KnockControl_Execute();
  } else {
    KnockControl_Disable(); /* Inhibited: input sensor faulty */
  }
}
```

FiM Configuration

Each FiM FunctionID references one or more DEM EventIDs. DemInhibitionMask selects which DTC status bits trigger inhibition. Typical: inhibit when testFailed bit is set (bit 0) or when confirmedDTC is set (bit 3).

CHAPTER 14

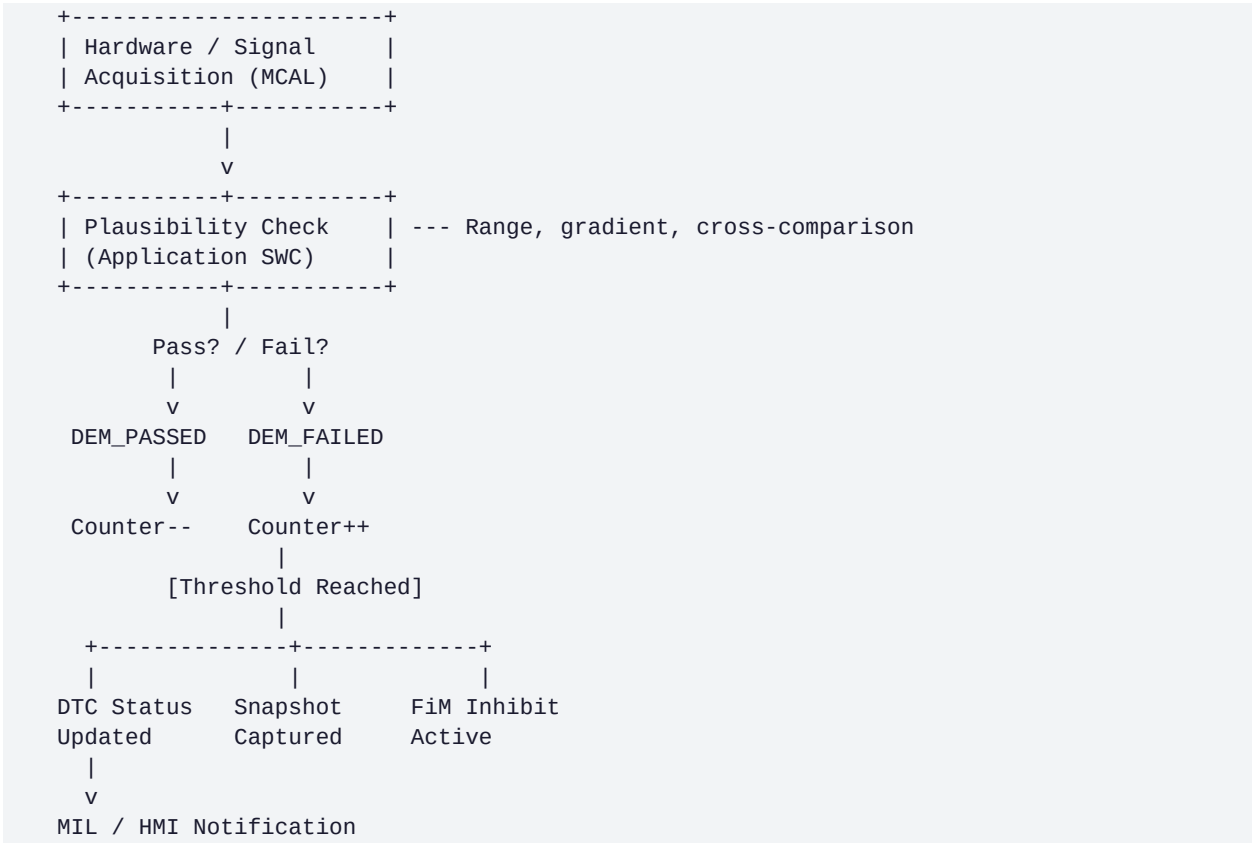
Fault Detection and Reaction

Fault detection in automotive ECUs operates at multiple abstraction levels, from hardware ADC range checks to complex cross-sensor plausibility monitors. ISO 26262 defines the requirements for safety-relevant fault monitoring.

14.1 Fault Detection Levels

Level	Mechanism	Examples	ISO Reference
Hardware Monitor	ADC out-of-range, SPI CRC, power supervisor	Voltage OOL, SPI frame error	ISO 26262 Part 5
MCAL / HW Abstraction	Driver-level plausibility, timeout	CAN bus-off, LIN timeout	ISO 26262 Part 6
BSW Monitor	OS watchdog, stack supervision	Stack overflow, CPU overload	ISO 26262 Parts 6/9
Application Monitor	Signal range, gradient, rational checks	Speed sensor out of range	ISO 26262 Part 6
Cross-System Monitor	E2E, redundant sensor comparison	Sensor disagreement > threshold	ISO 26262 Parts 5/9
Memory Monitor	CRC check, ECC, NvM CRC	Flash corruption, RAM bit-flip	ISO 26262 Part 5

14.2 Fault Detection Flow



14.3 Fault Reaction Strategies

Reaction Type	When Applied	Example	ASIL Level
Limp-Home Mode	Safety-critical sensor failure	Fixed RPM, reduced power on MAP fail	QM / ASIL A–D
Substitute Value	Non-safety path, plausibility fail	Model coolant temp from oil temp	QM
Function Inhibition (FiM)	Input invalid, function unsafe to run	Disable ACC if radar unavailable	ASIL A–D
Safe State Transition	ASIL-D safety goal threatened	Steering assist controlled ramp-off	ASIL D
Warning Only (MIL)	Emission fault, non-safety	MIL on, normal operation continues	OBD / QM
ECU Reset (WDG)	Severe internal fault	Watchdog timeout triggers hard reset	ASIL A–D
Graceful Degradation	Partial sensor failure	Reduce function scope, maintain core	ASIL B/C

14.4 Cross-Sensor Plausibility Monitor — Pseudocode

```
/* Wheel Speed Sensor Cross-Plausibility Monitor */
#define WSS_PLAUS_THRESHOLD_KMPH  5.0f
#define WSS_MIN_SPEED_FOR_CHECK  10.0f

void WssPlausMonitor_10ms(void) {
  float32 fl, fr, rl, rr;
  Rte_Read_WheelSpeedFL(&fl);  Rte_Read_WheelSpeedFR(&fr);
  Rte_Read_WheelSpeedRL(&rl);  Rte_Read_WheelSpeedRR(&rr);

  float32 ref = (fl + fr + rl + rr) / 4.0f;
  if (ref < WSS_MIN_SPEED_FOR_CHECK) return; /* skip at low speed */

  boolean fault = (fabsf(fl-ref) > WSS_PLAUS_THRESHOLD_KMPH) ||
                  (fabsf(fr-ref) > WSS_PLAUS_THRESHOLD_KMPH) ||
                  (fabsf(rl-ref) > WSS_PLAUS_THRESHOLD_KMPH) ||
                  (fabsf(rr-ref) > WSS_PLAUS_THRESHOLD_KMPH);

  Rte_Call_Dem_WSSPlaus_SetEventStatus(
    fault ? DEM_EVENT_STATUS_FAILED : DEM_EVENT_STATUS_PASSED);

  if (fault) FiM_SetPermission(FIM_FUNC_ABS, FALSE);
}
```

CHAPTER 15

Warning Lamps and Indicator Management

Warning indicators in modern vehicles communicate fault severity and system status to the driver. OBD-II mandates the MIL (Malfunction Indicator Lamp) for emission faults; safety and chassis systems use additional standardised lamps.

15.1 MIL — Malfunction Indicator Lamp (OBD-II Mandate)

- MIL illuminates within two OBD monitoring cycles (two drive cycles) after Type A or B fault confirmation.
- MIL extinguishes after three consecutive drive cycles without the fault being active (healing).
- MIL performs a 3-second bulb check at key-on (every ignition cycle).
- MIL cannot be manually cleared by the driver; only via OBD Mode 04 or UDS 0x14, or after automatic healing + aging.

15.2 Standard Warning Lamps in Modern Vehicles

Lamp	Colour/Symbol	DTC Relation	Standard/Regulation
MIL (Check Engine)	Amber engine symbol	Emission-related DTC confirmed (CDTC=1, WIR=1)	OBD-II / EOBD / ISO 15031
AWL (Amber Warning Lamp)	Amber triangle	WWH-OBD Class B/C faults	SAE J1939, ISO 27145
RSL (Red Stop Lamp)	Red stop symbol	Severe fault — immediate stop required	SAE J1939-73
BIL (Brake Indicator)	Red circle + P	ABS / ESC / EPB fault	ECE R13, FMVSS 135
TPMS Lamp	Tyre cross-section	Tyre pressure fault	FMVSS 138 / ECE R64
Battery/Charging	Battery symbol	Alternator / BMS fault	OEM specific
ADAS Status	Green/grey car+lines	ADAS mode (not fault)	SAE J3016 / ECE R157
EV Ready	Green power symbol	BMS operational state	ECE R100

15.3 MIL Control Logic — DEM Indicator API

```
/* DEM Configuration: Link DTC to MIL indicator */
/* ARXML: DemDtcAttributes -> DemIndicatorRef -> MIL */

/* Application MIL control task (10 ms) */
void MilControl_10ms(void) {
  Dem_IndicatorStatusType milStatus;
  Dem_GetIndicatorStatus(DEM_INDICATOR_MIL, &milStatus);

  switch (milStatus) {
    case DEM_INDICATOR_OFF:
      IoHwAb_SetOutput(IO_MIL, LOW); break;
    case DEM_INDICATOR_CONTINUOUS:
      IoHwAb_SetOutput(IO_MIL, HIGH); break;
    case DEM_INDICATOR_BLINKING:
      milTimer++;
      if (milTimer >= BLINK_PERIOD_COUNTS) milTimer = 0;
      IoHwAb_SetOutput(IO_MIL,
                       milTimer < BLINK_ON_COUNTS ? HIGH : LOW);
```

```
        break;
      default: break;
    }
  }
```

15.4 Permanent DTCs (OBD-II Mandate — Mode 0x0A)

Permanent DTCs (introduced CARB 2010) cannot be cleared by scan tool. They can only be removed when the ECU's own OBD monitor has run and confirmed PASSED:

Condition	Action on Permanent DTC
Tool clears DTCs (0x14 / Mode 04)	Confirmed DTC cleared, permanent DTC remains in Mode 0x0A
Monitor runs and PASSES once	Candidate for removal — not yet removed
Monitor passes two consecutive trips	Permanent DTC removed from memory
Battery disconnect	Permanent DTC must survive (NvM-backed)

CHAPTER 16

HMI — Human-Machine Interface for Diagnostics

HMI in diagnostics spans both in-vehicle driver interfaces (instrument cluster, head unit) and off-board tester tools (OBD readers, OEM dealer tools). The HMI layer translates raw DTC numbers and DID data into actionable information.

16.1 Off-Board Diagnostic Tools

Tool Type	Examples	Protocol	Use Case
OBD-II Generic Reader	BlueDriver, LAUNCH X431, Autel MaxiSys	OBD Mode 01–0A over CAN	Workshop, inspection
OEM Dealer Tool	ODIS (VW Group), ISTA (BMW), DealerExpert	UDS via DoIP/CAN	OEM full diagnostics
PDU API J2534	Drew Tech Mongoose, BOSCH VCI	ISO 22900 PDU API	OEM integration
Development Tool	ETAS INCA, Vector CANalyzer, ASAM CANape	UDS + XCP + OBD	Calibration, dev
EOL Test System	Softing, National Instruments, dSPACE	UDS Programming Session	End-of-line flash+cal
OTA Diagnostics	Uptane, AUTOSAR UCM	DoIP over HTTPS	Remote update / diag

16.2 ODX / PDX — Diagnostic Data Description

- ODX (Open Diagnostic Data eXchange, ISO 22901-1): XML-based format describing all ECU diagnostic capabilities — services, DIDs, DTCs, PIDs, coding parameters.
- PDX (Package Diagnostic eXchange, ISO 22901-3): Bundles ODX + firmware + calibration into a signed distribution package.
- SOVD (Service-Oriented Vehicle Diagnostics, ASAM SOVD V1.2): REST/HTTP-based diagnostic layer for HPC/Adaptive ECUs — ISO 22901-3 extension.
- UDX: Extends ODX with cybersecurity requirements (ISO 21434 integration) for secure diagnostic data exchange.

16.3 SAE J2534 — PassThru Interface

```
/* J2534 PassThru sequence — UDS ReadDataByIdentifier */

/* 1. Open interface device */
PassThruOpen(NULL, &deviceID);
PassThruConnect(deviceID, ISO15765, CAN_ID_BOTH, 500000, &channelID);

/* 2. Set CAN ID filter */
PASSTHRU_MSG mask    = { .Data={0xFF,0xFF,0xFF,0xFF}, .DataSize=4 };
PASSTHRU_MSG pattern = { .Data={0x00,0x00,0x07,0xE8}, .DataSize=4 };
PassThruStartMsgFilter(channelID, PASS_FILTER, &mask, &pattern, NULL, &fid);

/* 3. Send RDBI VIN request to ECU 0x7E0 */
PASSTHRU_MSG tx = {
  .ProtocolID=ISO15765, .TxFlags=ISO15765_FRAME_PAD, .DataSize=7,
  .Data={0x00,0x00,0x07,0xE0, 0x03,0x22,0xF1,0x90}
         /* CAN ID 0x7E0 */  /* SF: len=3, 0x22, F190 */
};
```

```
PassThruWriteMsgs(channelID, &tx, &numMsgs, 1000);

/* 4. Read 62 F1 90 + 17-byte VIN response */
PassThruReadMsgs(channelID, &rx, &numMsgs, 1000);
```

16.4 Python-UDS — Open-Source Tester Scripting

The udsoncan library (MIT-licensed) combined with python-can provides a full UDS client in Python. Ideal for CI/CD, HIL benches, and rapid tester prototyping without hardware dongles:

```
# pip install udsoncan python-can can-isotp
import udsoncan, can, isotp
from udsoncan.connections import PythonIsoTpConnection
from udsoncan.client import Client

bus   = can.Bus(interface="socketcan", channel="vcan0", bitrate=500000)
addr  = isotp.Address(isotp.AddressingMode.Normal_11bits, txid=0x7E0, rxid=0x7E8)
stack = isotp.CanStack(bus, address=addr)
conn  = PythonIsoTpConnection(stack)

with Client(conn, request_timeout=5) as client:
    # Enter extended session
    client.change_session(udsoncan.services.DiagnosticSessionControl
                          .Session.extendedDiagnosticSession)
    # Read VIN (DID 0xF190)
    r = client.read_data_by_identifier(0xF190)
    print("VIN:", r.service_data.values[0xF190])
    # Read all active DTCs
    dtcs = client.get_dtc_by_status_mask(0xFF)
    for d in dtcs.dtcs:
        print(f"  DTC {d.id:#08x}  status={d.status.byte:#04x}  severity={d.severity}")
    # Clear all DTCs
    client.clear_dtc(group=udsoncan.services.ClearDiagnosticInformation.allDTCGroup)
```

16.5 CANalyzer / CAPL Diagnostic Scripting

Vector CANalyzer with the Diagnostics option uses CAPL (Communication Access Programming Language) to automate UDS sequences on a real CAN network. CAPL scripts are compiled into .dll and executed at runtime:

```
/* CAPL: Read VIN + clear DTCs */
diagRequest ECU.ReadDataByIdentifier_VIN req;
diagRequest ECU.ClearDiagnosticInformation_AllDTCs clrReq;

on start {
  diagSetTarget("ECU");
  output(req);                          // 22 F1 90
}

on diagResponse ECU.ReadDataByIdentifier_VIN {
  char vin[18];
  diagGetPrimParam(this, "VehicleIdentificationNumber", vin, elcount(vin));
  write("VIN: %s", vin);
  output(clrReq);                       // 14 FF FF FF
}

on diagResponse ECU.ClearDiagnosticInformation_AllDTCs {
  write("DTCs cleared. Response: %02X", this.DiagResponse);
}
```

16.6 ODX / ASAM MCD-2D — Diagnostic Data Description Standard

ODX (Open Diagnostic data eXchange, ISO 22901-1) is the XML standard for describing all ECU diagnostic content: services, DIDs, DTCs, parameter encoding, session rules. ODX files feed tester tools (EDIABAS, CANdela, ISOLAR-EVE) enabling automatic generation of the tester-side stack:

ODX Element	ISO 22901-1 Container	Purpose
DIAG-LAYER	BaseVariant / EcuVariant	Top-level ECU description, inherits from base layer
DIAG-SERVICE	DIAG-COMMS	Service definition: SID, sub-function, request/response params
DTC-DOP	DIAG-DATA-PROPERTY	DTC symbolic name, fault path, status byte mapping
CODED-TYPE	DATA-OBJECT-PROP	Physical unit, encoding, byte position and scale for each DID param
STATE-CHART	STATE-CHARTS	Session and security level state transition machine (XML)
ECU-VARIANT-PATTERN	VEHICLE-INFO	VIN-based or DID-based variant selection for multi-ECU vehicles
FUNCTION-CLASS	FUNCTION-DICTIONARY	Logical grouping: identification, flash, calibration, fault readout

CHAPTER 17

Diagnostics Lifecycle

The diagnostics lifecycle spans the entire vehicle lifespan, from concept definition through production, after-sales service, and end-of-life. Each phase imposes specific diagnostic requirements and tooling.

17.1 Lifecycle Overview

```
+-------------------------------------------------------------------+
|                AUTOMOTIVE DIAGNOSTICS LIFECYCLE                   |
+----------+----------+----------+----------+-----------+----------+-----+
| Concept  | System   | Software | HW/SW    | Produc-   | After-   | EOL |
| Phase    | Design   | Dev      | Integr.  | tion      | Sales    |     |
+----------+----------+----------+----------+-----------+----------+-----+
| HARA     | DTC      | Monitor  | SIL/HIL  | EOL Flash | OBD Scan | DTC |
| Fault    | Planning | Coding   | Testing  | Coding    | Service  | dump|
| ISO 26262| SID/DID  | DCM/DEM  | Diag CAL | VIN prog  | Campaign | arc |
+----------+----------+----------+----------+-----------+----------+-----+
```

17.2 Development Phase Checklist

1. DTC Planning: Every FMEA/HARA fault path assigned DTC, debounce, and reaction parameters.
2. ODX Authoring: Diagnostic databases created for each ECU defining all DIDs, DTCs, PIDs, and coding.
3. AUTOSAR Configuration: DCM and DEM modules configured in ARXML with DID/DTC/RID/session/security entries.
4. RTE Generation: AUTOSAR toolchain generates RTE code connecting SWCs to DCM/DEM ports.
5. Monitor Implementation: Application SWCs implement fault monitors using DEM API.

17.3 Integration and Validation

Test Level	Activity	Tools Used
SIL (Software-in-Loop)	Unit test DTC logic, DCM responses, NRC handling	MATLAB/Simulink, CANoe simulation
MIL (Model-in-Loop)	Simulate monitors with injected fault inputs	Simulink, INCA, Beckhoff
HIL (Hardware-in-Loop)	Real ECU + bus + simulated plant model	dSPACE SCALEXIO, NI PXI
Integration Bench	Full vehicle network diagnostics validation	CANalyzer, INCA on network bench
Vehicle Test	OBD drive cycles, readiness monitor completion	Real vehicle, CARB OBD drive cycle
Cybersecurity Test	Penetration test of diagnostic interfaces	Keysight, Synopsys, IOActive

17.4 End-of-Line (EOL) Production Diagnostics

EOL Step	UDS Service Used	Purpose
1. Establish Connection	`0x10 ExtendedSession + 0x27 SecurityAccess`	Open ECU for programming

EOL Step	UDS Service Used	Purpose
2. Flash ECU Software	`0x28 CommControl + 0x34/0x36/0x37`	Program application + calibration
3. Write VIN	`0x2E DID 0xF190`	Programme unique VIN into NvM
4. ECU Variant Coding	`0x2E OEM-specific DID`	Configure vehicle options
5. Calibration Routines	`0x31 RID (OEM-specific)`	SAS, AFM, fuel trim calibration
6. Clear DTCs	`0x14 ClearAllDTC`	Remove faults generated during EOL
7. OBD Readiness Check	`0x01 PID 0x01`	Verify readiness monitors OK
8. ECU Reset	`0x11 hardReset`	Restore to operational state

17.5 OTA Diagnostics (2025–2026)

Capability	Standard/Framework	Description
Remote DTC Readout	ISO 14229-5 UDS on DoIP	Fleet ECU DTC monitoring via cloud
OTA Software Update (FOTA)	AUTOSAR UCM + Uptane	Signed FOTA with rollback and verification
Remote Security Patch	ISO 21434 + VSOC	Vulnerability patches via OTA campaign
Remote Diagnostic Access	ISO 13400 DoIP + TLS 1.3	Dealer-level UDS via cloud-secured tunnel
Prognostic Data Analytics	DEM trend + cloud ML	Predictive maintenance from DEM data
SBOM Traceability	ISO 5230 / NTIA SBOM	Full SW component traceability for CVE

CHAPTER 18

ECU Programming and Flash Management

ECU programming (firmware update) uses UDS Programming Session with a standardised multi-step sequence. Any deviation from the required sequence generates NRC 0x24 (requestSequenceError).

18.1 Programming Session Service Sequence

Step	Service	Purpose
1	0x10/0x02 – ProgrammingSession	Enter programming session
2	0x27/0x01, 02 – SecurityAccess	Unlock programming security level
3	0x28/0x03 – CommunicationControl	Disable normal bus Rx/Tx
4	0x85/0x02 – ControlDTCSetting	Disable DTC storage
5	0x31/0x01/0xFF00 – RoutineControl	Erase target flash sectors
6	0x34 – RequestDownload	Specify address, size, compression
7	0x36 (N×) – TransferData	Stream firmware blocks
8	0x37 – RequestTransferExit	End transfer, CRC verify
9	0x31/0x01/0xFF01 – CheckProgrammingDependencies	Validate SW consistency
10	0x28/0x00 – CommunicationControl	Re-enable normal Rx/Tx
11	0x85/0x01 – ControlDTCSetting	Re-enable DTC storage
12	0x11/0x01 – ECUReset	Hard reset to boot new SW

18.2 RequestDownload and TransferData Protocol

```
/* RequestDownload (0x34) */
Tester TX: [0x34]
           [0x00]             -- dataFormatIdentifier (no compress)
           [0x44]             -- 4-byte address + 4-byte length
           [0x00][0x40][0x00][0x00] -- memoryAddress = 0x00400000
           [0x00][0x20][0x00][0x00] -- memorySize = 2 MB

ECU    RX: [0x74]
           [0x40]             -- lengthOfMaxBlockSize = 4 bytes
           [0x00][0x04][0x00][0x00] -- maxBlockLength = 1024 bytes

/* TransferData (0x36) — repeated for each 1024-byte block */
Tester TX: [0x36][blockSeqCtr] [1024 bytes firmware data]
ECU    RX: [0x76][blockSeqCtr] -- positive response
           -- blockSeqCtr: 0x01, 0x02, ..., 0xFF, 0x00, 0x01...

/* RequestTransferExit (0x37) */
Tester TX: [0x37]
ECU    RX: [0x77]  -- transfer complete, SW CRC verified
```

NRC 0x78 During Flash

Flash erase operations can take up to 5 seconds per sector. The ECU sends NRC 0x78 (RequestCorrectlyReceived-ResponsePending) to reset the P2* timer. Testers MUST restart their P2*

timeout (5000 ms default) on each 0x78 received.

18.3 Flash Block Integrity Verification

After RequestTransferExit (0x37), the ECU performs CRC or cryptographic hash verification before activating the new SW. OEMs require at minimum CRC-32 and for EVITA FULL ECUs: SHA-256 + RSA/ECDSA backed by the HSM:

```
/* Integrity check flow after 0x37 RequestTransferExit */
  STEP 1: CRC-32 over entire flashed block
    crc_calc = CRC32_Calculate(FLASH_BASE_ADDR, block_len);
    if (crc_calc != expected_crc_from_header) -> NRC 0x70

  STEP 2: SHA-256 hash verification (if OEM-required)
    Crypto_Hash(CRYPTO_ALGOID_SHA256, flash_data, hash_out);
    if (memcmp(hash_out, sw_manifest_hash, 32) != 0) -> NRC 0x70

  STEP 3: RSA-2048 / ECDSA-P256 signature check (HSM-backed)
    Csm_SignatureVerify(job_id, sig, sig_len, hash_out, 32);
    if (result != CRYPTO_E_VER_OK) -> NRC 0x70

  STEP 4: Anti-rollback version check
    if (new_sw_version < otp_min_version) -> NRC 0x70

  STEP 5: Write programming fingerprint (DID 0xF15A)
    NvM_WriteBlock(FINGERPRINT_NVM_ID, &fingerprint_data);
```

18.4 Programming Fingerprint, Rollback Protection & Dual-Bank Strategy

Mechanism	DID / Service	Description
Programming Counter	0xF1A0	Monotonic write counter stored in OTP/NvM — can never decrease
Programming Date/Time	0xF18B	ISO 8601 UTC timestamp of last programming event
Programming Tester SN	0xF15A	Serial number of tester device performing flash (WDBI at EOL)
SW Fingerprint Record	0xF15B	Last 3 programming records stored by ECU for traceability
Anti-Rollback Version	HSM OTP fuse	Minimum allowed SW version burned into one-time-programmable fuses
Rollback Trigger	NRC 0x70 / reset	Failed verify: ECU boots fallback SW from redundant flash bank
Dual-Bank Flash	HW partition	Active bank + inactive bank; swap atomically on verified programming
Secure Boot	HSM at startup	HSM measures SW hash on every power-on; blocks untrusted boot

ISO 26262 Programming Safety

Safety-relevant ECUs (ASIL-B and above) must guarantee that a power-loss or watchdog reset during programming does not leave the ECU in a non-functional state. Dual-bank flash, hardware watchdog management (WdgM_SetMode), and VBatt monitoring (below 11V abort) are mandatory in most OEM programming specifications.

CHAPTER 19

Diagnostics and Cybersecurity (ISO 21434)

ISO 21434:2021 mandates Threat Analysis and Risk Assessment (TARA) for all diagnostic interfaces. The OBD-II port and UDS-over-DoIP are primary automotive attack vectors and must be secured throughout the product lifecycle.

19.1 Threat Landscape for Diagnostic Interfaces

Threat	Attack Vector	Potential Impact	TARA Severity
Unauthorised ECU Flash	UDS 0x34/0x36 without auth	Malicious firmware installation	Critical (S3)
DTC Clearing Abuse	UDS 0x14 / OBD Mode 04	Hide fault before inspection	Significant (S2)
Security Access Bypass	Seed/key algorithm reversing	Full UDS write/control access	Critical (S3)
DoIP Exploitation	IPv6/Ethernet DoIP gateway	Remote ECU access from internet	Critical (S3)
CAN Bus Eavesdropping	OBD port tap	ECU fingerprinting	Moderate (S1)
Replay Attack	Captured UDS frames replayed	Reuse old seed/key pair	Significant (S2)
DoS via CAN Flooding	CAN bus flooding at OBD port	Bus-off, ECU unresponsive	Significant (S2)

19.2 UDS Authentication Service (0x29) — ISO 14229-1:2020

Service 0x29 (Authentication) replaces basic seed-key with certificate-based or HMAC challenge-response authentication backed by HSM:

Sub-function	Value	Description
deAuthenticate	`0x00`	Remove all granted access levels
verifyCertificateUnidirectional	`0x01`	ECU verifies tester certificate (one-way)
verifyCertificateBidirectional	`0x02`	Mutual certificate verification
proofOfOwnership	`0x03`	Challenge-response ownership proof
transmitCertificate	`0x04`	Send certificate chain to ECU
requestChallengeForAuthentication	`0x05`	Request nonce/challenge from ECU
verifyProofOfOwnershipUnidirectional	`0x06`	ECU verifies HMAC/signature
authenticationConfiguration	`0x07`	Read authentication capabilities

19.3 Diagnostic Security Hardening Measures

- HSM-backed Security Access: Use automotive HSM (SHE, Infineon HSM, Aurix HSM) for key storage and seed-key derivation.
- Certificate-based 0x29 Authentication for all programming and calibration access.
- Delay after failed attempts: NRC 0x37 (requiredTimeDelayNotExpired) enforces lockout after max failed attempts.

- S3 timer: Automatic session exit if no diagnostic activity within 5 seconds.
- DoIP TLS 1.3: ISO 13400-2:2019 includes TLS 1.3 mutual authentication for remote access.
- OBD port access control: Physical port authentication or selective port disable for sensitive modes.
- TARA integration: All diagnostic interfaces must be included in ISO 21434 TARA with documented risk treatment.

19.4 SecOC — Secure On-Board Communication & Diagnostic Implications

ISO 21434 and AUTOSAR SecOC (Secure On-Board Communication) authenticate inter-ECU signals on CAN/CAN-FD using a Freshness Value (FV) and Message Authentication Code (MAC). Diagnostic interactions with SecOC-protected ECUs require authentication-aware request routing:

```
/* SecOC frame structure on CAN */
+----------------+-----------+----------------+
| Payload (Data) | Freshness | Truncated MAC  |
| N bytes        | Counter   | (e.g. 24-bit)  |
+----------------+-----------+----------------+

/* Diagnostic impact: */
// 1. UDS requests to SecOC-protected ECUs routed via trusted gateway
// 2. Gateway adds SecOC protection for forwarded diagnostic messages
// 3. Service 0x84 (SecuredDataTransmission) wraps UDS in crypto envelope
// 4. OBD II tester tools bypass SecOC via physical DLC (direct link)

/* AUTOSAR SecOC ARXML key containers */
  SecOCDeployment       // Binds SecOC to I-PDU
  SecOCFreshnessValueId // Monotonic counter reference
  SecOCAuthInfoTxLength // Truncated MAC length (16-96 bits)
  SecOCDataId           // Unique ID for CMAC computation input
```

19.5 Diagnostic Access Control Matrix

Security-sensitive ECUs require a documented Access Control Matrix mapping each UDS service to the required session, security level, and authentication certificate:

Service	Default Session	Extended Session	Programming Session	Notes
0x22 RDBI (VIN, status)	Yes — no auth	Yes — no auth	Yes — no auth	Open read for regulatory compliance
0x22 RDBI (key material)	No	Yes — Sec Level 1	No	Key status only; not key value
0x2E WDBI (calibration)	No	Yes — Sec Level 2	No	OEM-specific calibration DIDs
0x2E WDBI (VIN/coding)	No	No	Yes — Sec Level 1	EOL programming only
0x2F IOCBI	No	Yes — Sec Level 2	No	Actuator test; speed interlock required
0x31 RID (check dep.)	No	No	Yes — Sec Level 1	Part of flash sequence
0x34/0x36/0x37 Flash	No	No	Yes — Sec Level 1	OEM signed firmware required
0x14 ClearDTC	Yes	Yes	Yes	Functional address; any session
0x29 Authentication	Yes — to get cert	Yes	Yes	ISO 14229-1:2020, certificate chain

CHAPTER 20

Practical Implementation Examples

This chapter provides end-to-end implementation examples demonstrating the complete chain from sensor acquisition to DTC storage, UDS service dispatch, and OBD PID processing.

20.1 Complete DTC Monitor — Engine Coolant Temperature

```
/* ============================================================
   USE CASE: ECT Sensor Out-Of-Range (DTC P0117)
   Event ID:  DEM_EVENT_ID_ECT_LOW
   Period:    10 ms task
   Debounce:  Counter-based, threshold +5/-5
   Reaction:  Substitute value 90 degC + MIL ON
   ============================================================ */

#define ECT_ADC_MIN       0.1f   /* V - short to gnd */
#define ECT_ADC_MAX       4.9f   /* V - open circuit */
#define ECT_SUBSTITUTE_C  90.0f  /* substitute in degC */

void EctMonitor_10ms(void) {
  float32 adcVolts;
  Rte_Read_EctAdc_AdcValue(&adcVolts);

  boolean inRange = (adcVolts >= ECT_ADC_MIN && adcVolts <= ECT_ADC_MAX);

  if (!inRange) {
    Rte_Call_DiagMonitor_ECT_SetEventStatus(DEM_EVENT_STATUS_FAILED);
    Rte_IWrite_EctSubstitute_CoolantTemp(ECT_SUBSTITUTE_C);
    Rte_IWrite_EctSubstitute_IsSubstitute(TRUE);
  } else {
    Rte_Call_DiagMonitor_ECT_SetEventStatus(DEM_EVENT_STATUS_PASSED);
    Rte_IWrite_EctSubstitute_IsSubstitute(FALSE);
  }
}
```

20.2 Minimal UDS Service Dispatcher

```
/* Minimal UDS dispatcher — table-driven (DCM-like) */
typedef Std_ReturnType (*UdsSvcHandler_t)(
  const uint8* req, uint16 reqLen, uint8* rsp, uint16* rspLen);

typedef struct { uint8 sid; UdsSvcHandler_t fn; } UdsEntry_t;

static const UdsEntry_t UDS_TABLE[] = {
  { 0x10, Uds_DiagnosticSessionControl },
  { 0x11, Uds_ECUReset               },
  { 0x14, Uds_ClearDTC               },
  { 0x19, Uds_ReadDTCInformation     },
  { 0x22, Uds_ReadDataByIdentifier   },
  { 0x27, Uds_SecurityAccess         },
  { 0x2E, Uds_WriteDataByIdentifier  },
  { 0x31, Uds_RoutineControl         },
  { 0x3E, Uds_TesterPresent          },
};

void Uds_Dispatch(const uint8* rx, uint16 rxLen,
```

```
                uint8* tx, uint16* txLen) {
  uint8 sid = rx[0];
  for (uint8 i = 0; i < ARRAY_SIZE(UDS_TABLE); i++) {
    if (UDS_TABLE[i].sid == sid) {
      if (UDS_TABLE[i].fn(rx, rxLen, tx, txLen) != E_OK)
        Uds_SendNRC(tx, txLen, sid, DCM_E_GENERALREJECT);
      return;
    }
  }
  Uds_SendNRC(tx, txLen, sid, DCM_E_SERVICENOTSUPPORTED);
}
```

20.3 OBD PID Supported Bitmask

```
/* OBD Mode 01 PID 0x00 response builder */
static const uint8_t SUPPORTED_PIDS[] = {
  0x04, 0x05, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10, 0x11
};

void Obd_BuildPidBitmask(uint8 startPid, uint8* bitmask) {
  memset(bitmask, 0, 4);
  for (int i = 0; i < ARRAY_SIZE(SUPPORTED_PIDS); i++) {
    uint8 pid = SUPPORTED_PIDS[i];
    if (pid >= startPid+1 && pid <= startPid+0x20) {
      uint8 bitPos = (startPid + 0x20) - pid;
      bitmask[bitPos/8] |= (1 << (7 - (bitPos % 8)));
    }
  }
}

/* Usage: Request PID 0x00 response = supported PIDs bitmask */
/* bitmask[3:0] represents PIDs 0x01..0x20 in MSB-first order */
```

20.4 DEM NvM Block Layout

NvM Block	Content	Size (typical)	Write Trigger
DEM_PRIMARY_ENTRY_N	DTC number, status byte, occurrence count, aging counter	12 bytes	On status change
DEM_SNAPSHOT_RECORD_N	Freeze frame data (configurable PIDs/DIDs)	32–64 bytes	On DTC confirmation
DEM_EXTENDED_DATA_N	Trip counter, mileage at fault, timestamp	16 bytes	On event qualification
DEM_PERMANENT_DTC_N	Permanent DTC list (OBD emission)	8 bytes per DTC	On confirmation
DEM_INDICATOR_STATUS	MIL/AWL indicator state bitmap	4 bytes	On indicator change
DEM_OBD_FREEZE_FRAME	OBD Mode 02 freeze frame data	40 bytes	On first confirmed DTC

CHAPTER 21

AUTOSAR Adaptive and SDV Diagnostics (2024–2026)

AUTOSAR Adaptive Platform (AP) targets high-performance ECUs for ADAS, central compute, and connectivity running on multicore POSIX SoCs. The diagnostic stack fundamentally differs from Classic AUTOSAR through service-oriented C++ APIs and native Ethernet transport.

21.1 Classic vs. Adaptive AUTOSAR Diagnostics

Feature	AUTOSAR Classic	AUTOSAR Adaptive
Execution model	Fixed RTOS task scheduling	POSIX processes / ara::exec
Diagnostic module	DCM BSW module (C)	ara::diag service (C++17)
DEM equivalent	DEM BSW module	ara::diag DTC/snapshot APIs
Transport	CAN TP, DoIP via ComStack	DoIP native Ethernet / SOME/IP
Configuration	ARXML + static code gen	ARXML + Service Manifest
Security	SecurityAccess (0x27)	`0x29 Certificate Auth + TLS 1.3`
OTA Updates	Limited NvM writes	Full FOTA via ara::ucm
Language	C / MISRA-C	C++14/17, ara::core

21.2 ara::diag — Key APIs (AUTOSAR AP R24-11)

```
// ara::diag DM (Diagnostic Manager) — C++17 examples

// Register DID 0x1234 read handler
auto didHandler = ara::diag::DataIdentifier::Create(0x1234);
didHandler->RegisterGetHandler(
  [](ara::diag::MetaInfo& meta,
     ara::diag::CancellationHandler& cancel)
     -> ara::diag::OperationOutput {
       ara::diag::OperationOutput out;
       out.responseData = GetBatteryStateVector();
       return out;
  }
);

// Report DTC (equivalent of Dem_SetEventStatus)
auto event = ara::diag::Event::Create(
  ara::diag::EventId{0x040500}); // DTC

// Fault detected
event->ReportIgnoreCancellation(
  ara::diag::EventStatusByte::kFailed);

// Fault healed
event->ReportIgnoreCancellation(
  ara::diag::EventStatusByte::kPassed);
```

21.3 DoIP — Diagnostics over IP (ISO 13400-2)

DoIP Message Type	Payload Type ID	Description
VehicleIdentificationRequest	0x0001	Broadcast — discover ECUs on Ethernet network
VehicleIdentificationResponse	0x0004	Returns VIN, EID, GID, current IP address
RoutingActivationRequest	0x0005	Establish DoIP logical diagnostic connection
RoutingActivationResponse	0x0006	Confirm or deny routing activation
DiagnosticMessage	0x8001	UDS request: Source Address + Target Address + UDS bytes
DiagnosticMessagePositiveAck	0x8002	Message received and forwarded to target ECU
DiagnosticMessageNegativeAck	0x8003	Invalid SA/TA or routing not activated
AliveCheckRequest	0x0007	Keep-alive ping from tester to maintain session
AliveCheckResponse	0x0008	ECU confirm alive to tester

21.4 AUTOSAR UCM — OTA Software Update Flow

```
Cloud Backend (Uptane OTA Server)
       |
       | HTTPS + Uptane signed package
       v
Telematics / OTA Client ECU
       |
       | DoIP + UDS 0x34/0x36/0x37 or ara::ucm API
       v
Target ECU (Adaptive Platform)
  UCM receives and verifies package
       |
       | Uptane root + targets metadata check
       v
  Software Cluster Manifest verified
       |
       v
  UCM activates new SW version
  (or rolls back on verification failure)
       |
       v
  ara::exec restarts affected processes
  DTC cleared, UCM DID updated
```

21.5 SOVD — Service-Oriented Vehicle Diagnostics (ASAM V1.2:2024)

SOVD (Service-Oriented Vehicle Diagnostics) is an ASAM-standardized REST/HTTP API designed for High-Performance Computers (HPCs) running AUTOSAR Adaptive or Linux. It replaces the binary UDS protocol with human-readable JSON over HTTP/2, enabling SDV diagnostics from standard browsers, cloud backends, or microservices:

SOVD Concept	UDS Equivalent	SOVD Endpoint Example
Capability Discovery	DSC / supported SIDs	GET /components/{id}/capabilities
Read Data	RDBI (0x22)	GET /data/{did_name}
Write Data	WDBI (0x2E)	PATCH /data/{did_name} body: {"value": ...}
Fault Management	ReadDTCInfo (0x19)	GET /faults ?status=active
Clear Faults	ClearDTC (0x14)	DELETE /faults
Routines	RoutineControl (0x31)	POST /procedures/{rid}

SOVD Concept	UDS Equivalent	SOVD Endpoint Example
Software Update	RequestDownload/ TransferData	POST /updates (streaming)
Logging / Tracing	No UDS equivalent	GET /logs?since=2026-01-01T00:00Z (DLT log stream)
Lock (Access Control)	SecurityAccess (0x27)	POST /lock body: {"mode":"exclusive"}

```
/* SOVD REST example — Python requests — Read ECU software version */
import requests

BASE = "http://192.168.0.10:8080/v1/components/PowertrainECU"
headers = {"Authorization": "Bearer <JWT_TOKEN>", "Content-Type": "application/json"}

# Read DID F189 (ECU Software Version)
r = requests.get(f"{BASE}/data/EcuSoftwareVersion", headers=headers)
print(r.json())   # {"EcuSoftwareVersion": "1.4.2-build.20260101"}

# Get active faults
r = requests.get(f"{BASE}/faults", params={"status": "active"}, headers=headers)
for fault in r.json()["faults"]:
    print(fault["id"], fault["description"], fault["status"])

# Clear all faults
r = requests.delete(f"{BASE}/faults", headers=headers)
print(r.status_code)  # 204 No Content on success
```

21.6 CAN FD Diagnostics — 64-Byte Frame Benefits

CAN FD (ISO 11898-1:2015) raises the payload from 8 to 64 bytes and the data-phase bit rate to up to 8 Mbit/s. Key diagnostic benefits:

Aspect	Classical CAN (ISO-TP)	CAN FD (ISO 15765-2:2016+)
Max payload per frame	8 bytes	64 bytes — reduces frame count up to 8x
Max transfer size	4095 bytes (12-bit DL)	2^32-1 bytes with extended addressing
Flash throughput (typical)	~20 KB/s @ 500 kbps	~400 KB/s @ 2 Mbit/s data-phase
ISO-TP overhead	1-2 bytes PCI per frame	Same PCI structure; fewer total frames
DID read efficiency	Multiple SFs for large DIDs	Large DIDs fit in single 64-byte frame
AUTOSAR transport	CanTp module	CanTp module with FD extensions (AUTOSAR R22-11+)
OEM adoption (2026)	All ECUs	Powertrain, ADAS, central GW, OTA-capable ECUs

Appendix A: UDS Timing Parameters (ISO 14229-2)

Parameter	Symbol	Default	Description
P2 Server	P2Server_max	50 ms	Max time from end of request to start of response
P2* Server	P2StarServer_max	5000 ms	Extended max response time after NRC 0x78
S3 Server	S3Server_min	5000 ms	Non-default session timeout without activity
P3 Client	P3Client_max	5000 ms	Max inter-message gap on client side
P4 Client	P4Client_min	0 ms	Min gap between client messages

Appendix B: Acronym Glossary

Acronym	Full Form
AP	AUTOSAR Adaptive Platform
ASIL	Automotive Safety Integrity Level (ISO 26262)
AUTOSAR	AUTomotive Open System ARchitecture
BSW	Basic Software (AUTOSAR)
CARB	California Air Resources Board
CDTC	Confirmed DTC (DTC status byte bit 3)
DCM	Diagnostic Communication Manager (AUTOSAR BSW)
DEM	Diagnostic Event Manager (AUTOSAR BSW)
DID	Data Identifier (16-bit, UDS 0x22/0x2E)
DLC	Data Link Connector (16-pin OBD-II port)
DoIP	Diagnostics over Internet Protocol (ISO 13400)
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
EOBD	European On-Board Diagnostics
EOL	End-of-Line (production-line testing)
FDC	Fault Detection Counter (-128..+127)
FiM	Function Inhibition Manager (AUTOSAR)
FMEA	Failure Mode and Effects Analysis
FOTA	Firmware Over-the-Air
HARA	Hazard Analysis and Risk Assessment (ISO 26262)
HIL	Hardware-in-the-Loop
HMI	Human-Machine Interface
HSM	Hardware Security Module
MIL	Malfunction Indicator Lamp ("Check Engine")
MCAL	Microcontroller Abstraction Layer (AUTOSAR)
NRC	Negative Response Code
NvM	Non-volatile Memory Manager (AUTOSAR)
OBD	On-Board Diagnostics
ODX	Open Diagnostic Data eXchange (ISO 22901)
OEM	Original Equipment Manufacturer
OTA	Over-the-Air
PDTC	Pending DTC (DTC status byte bit 2)

Acronym	Full Form
PID	Parameter Identifier (OBD, 8-bit)
RID	Routine Identifier (UDS 0x31)
RTE	Runtime Environment (AUTOSAR)
SDV	Software-Defined Vehicle
SID	Service Identifier (UDS)
SIL	Software-in-the-Loop
SPRMIB	suppressPosRspMsgIndicationBit (bit 7 of sub-function)
TARA	Threat Analysis and Risk Assessment (ISO 21434)
UCM	Update and Configuration Management (AUTOSAR AP)
UDS	Unified Diagnostic Services (ISO 14229)
VIN	Vehicle Identification Number
WDBI	WriteDataByIdentifier (UDS 0x2E)
WIR	Warning Indicator Requested (DTC status byte bit 7)
WWH-OBD	World-Wide Harmonized OBD (ISO 27145)

Appendix C: Key Standards Reference

Standard	Title	Year
ISO 14229-1:2020	UDS — Application Layer	2020
ISO 14229-2:2013	UDS — Session Layer (Timing)	2013
ISO 14229-3:2012	UDS on CAN	2012
ISO 14229-5:2013	UDS on Internet/IP (DoIP)	2013
ISO 14229-8:2020	UDS on CAN-FD	2020
ISO 15031-5:2015	OBD — Emission-Related Diagnostic Services	2015
ISO 15765-2:2016	CAN Transport Protocol (ISO-TP)	2016
ISO 13400-2:2019	DoIP — Transport Layer	2019
SAE J1979-2:2022	OBD-II Services (updated PID set)	2022
SAE J2012-DA:2020	DTC Classification	2020
SAE J2534-1:2019	PassThru Programming Interface	2019
ISO 22901-1:2008	ODX Open Diagnostic Data Exchange	2008
ISO 26262:2018	Functional Safety — Road Vehicles	2018
ISO 21434:2021	Cybersecurity Engineering — Road Vehicles	2021
ISO 27145-4:2012	WWH-OBD Application Layer	2012
AUTOSAR R24-11	Classic + Adaptive Platform Release (Nov 2024)	Nov 2024
SAE J1939-73:2022	Heavy Duty Diagnostics Layer	2022
ASAM SOVD V1.2:2024	Service-Oriented Vehicle Diagnostics	2024